MedraLink

Decentralized Healthcare Data Interoperability & Audit Provenance Platform

MASTER TECHNICAL REPORT BOOK & SYSTEM SPECIFICATION

Competition Track	Blockchain Olympiad Bangladesh (BCOLBD 2026) — Student Track
Team ID	6a7f5f0a67368
Institution	University of Information Technology and Sciences (UITS), Dhaka
Team Lead	Kazi Md Azhar Uddin Abeer (azhar_uddin1120@uits.edu.bd / +8801923156653)
Team Members	Ahmad Abdali Khan (ahammed911khan@gmail.com) & Sumaia Bintey Ismail
Date & Status	25 June 2026 • Official Project Technical Dossier (v1.0.0-PROTOTYPE)

Table of Contents

Executive Summary & Master Overview

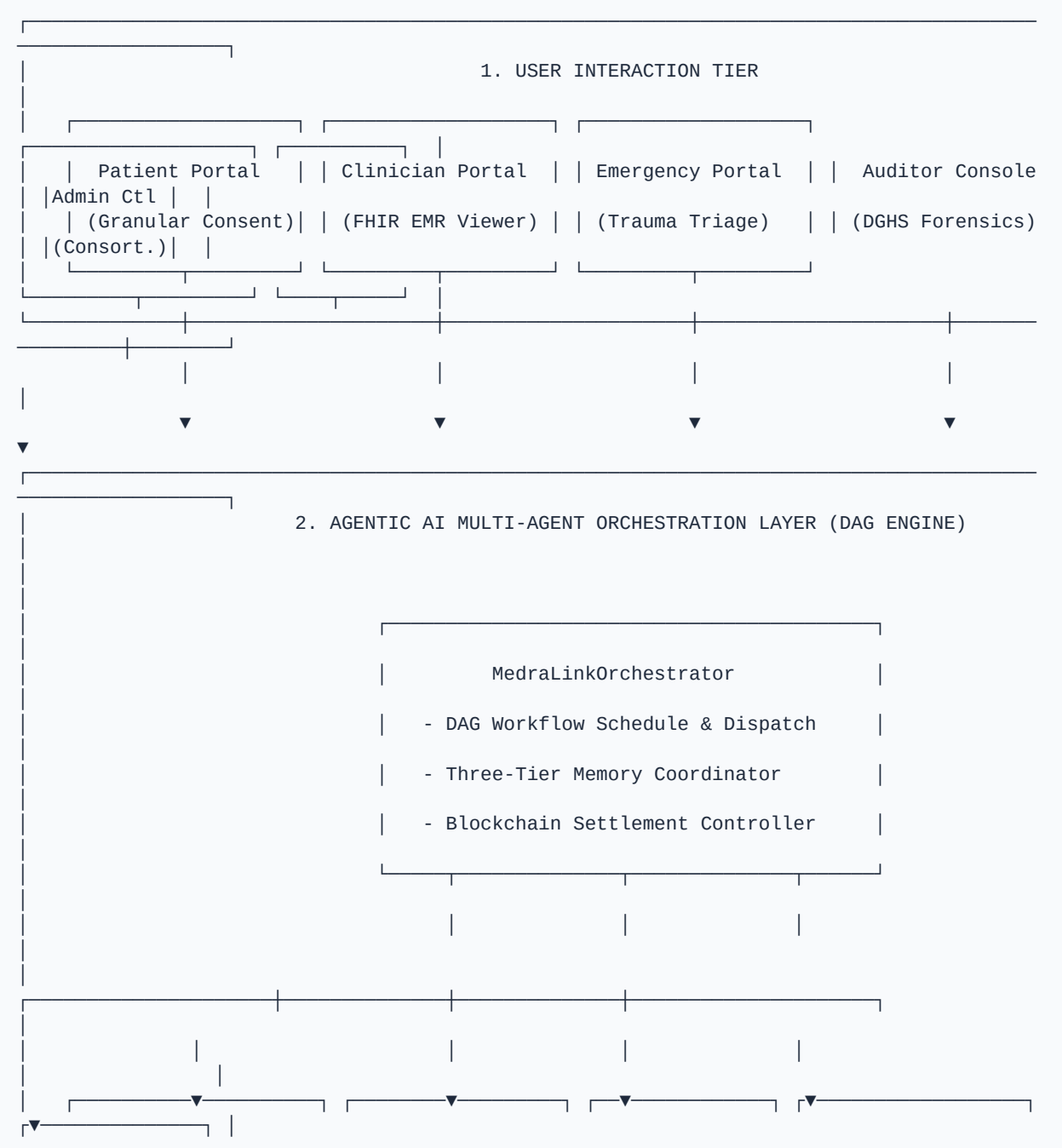
Competition: Blockchain Olympiad Bangladesh (BCOLBD 2026)

Project: MedraLink — Decentralized Healthcare Data Interoperability & Audit Provenance Platform

Master Agent Manual: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)

Track: Student Category — Blockchain / Agentic AI Track

🔬 Master System X-Ray Blueprint




ConsentAgent
- Dynamic PDPO Rules
- Purpose Binding
- Temporal Expiry
- Fail-Closed Gate
FHIRAgent
- SNOMED Bindings
- LOINC Lab Code
- RxNorm Dosage
- FHIR R4 Bundle
EmergencyTriage
- GCS/MAP Calc
- Shock Index
- 60-min Token
- ESI Level 1
AuditAgent
- Block Scanner
- Anomaly Detector
- Hash Continuity
IdentityAdapt.
- Porichoy NID
- Salted Hash
- Zero-PII Gen
- Ref Derived
3. THREE-TIER MEMORY HIERARCHY LAYER
Tier 1: Session Memory
- Ephemeral Node.js Context
- Active Clinician Identity
- Intermediate DAG Payloads
Tier 2: Vector Knowledge
- SNOMED-CT / LOINC / RxNorm
- Clinical Practice Rules
- PDPO 2025 Legal Statutes
Tier 3: Ledger State
- World State (CouchDB)
- Block Hash Store
- Composite Key Index
gRPC / REST
Encrypted I/O
4. PERMISSIONED BLOCKCHAIN (Hyperledger Fabric 2.5)
- Channel: `medralink-main`
- 9 Canonical Smart Contract Transactions (Go)
- 3-Node Raft Crash Fault Tolerant Consensus Cluster
- Cryptographic Proof Anchors: recordHash, patientHash
5. OFF-CHAIN ENCRYPTED CLINICAL STORE
- Custodial Hospital Cloud Vault
- AES-256-GCM Envelope Encryption (DEK)
- Plaintext Zero-PII FHIR Payloads
- Storage Pointer: s3://vault/rec.enc

📚 Complete Technical Documentation Suite

Document	Purpose & Contents	Primary Audience
📋 [`Medralink_Prototype_Execution_Plan_v1.0.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/Medralink_Prototype_Execution_Plan_v1.0.md)	Master 12-day execution roadmap, self-evaluated against the BCOLBD scoring rubric (Score: 90/100).	Evaluators & Team
🤖 [`AGENTIC_AI_ARCHITECTURE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/AGENTIC_AI_ARCHITECTURE.md)	Autonomous 5-Agent Architecture (`ConsentAgent`, `FHIRAgent`, `EmergencyTriageAgent`, `AuditAgent`, `MedraLinkOrchestrator`), DAG Execution Model, Inter-Agent Message Bus & 3-Tier Memory.	AI & System Evaluators
🏛️ [`ARCHITECTURE_SPECIFICATION.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/ARCHITECTURE_SPECIFICATION.md)	Deep dive into multi-tier topology X-Ray, Raft consensus block commit pipeline, node topologies, and on-chain / off-chain cryptographic separation.	Technical Evaluators
📜 [`CHAINCODE_SPECIFICATION.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/CHAINCODE_SPECIFICATION.md)	Technical contract specification, composite key trie graph, state machine, deterministic timestamps, and validation pipeline for all **9 Canonical Smart Contract Transactions** in Go.	Blockchain Judges
🏥 [`FHIR_INTEROPERABILITY_GUIDE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/FHIR_INTEROPERABILITY_GUIDE.md)	HL7 FHIR R4 semantic normalization X-Ray, terminology bindings (SNOMED-CT, LOINC, RxNorm), 6 core resource types, and AES-256-GCM authenticated encryption.	Healthcare IT Judges
🛡️ [`GOVERNANCE_AND_SECURITY_MANUAL.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/GOVERNANCE_AND_SECURITY_MANUAL.md)	3-Pillar Governance Triad X-Ray, PDPO 2025 compliance (§12 & §24), tokenless rationale, and emergency break-glass accountability loop.	Governance Judges
🎬 [`DEMO_SCRIPT.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/DEMO_SCRIPT.md)	Step-by-step 9-stage live demonstration walkthrough & execution pipeline matching Whitepaper Section G and the interactive Demo Tour modal.	Pitch Presenters

🧪 [`TESTING_AND_QA_REPORT.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/TESTING_AND_QA_REPORT.md)	Automated unit and integration test reports (100% test pass rate across **34 automated tests: 9 Go Smart Contract tests + 25 Node.js API integration tests**).	QA & Code Auditors
🏆 [`COMPETITION_PRESENTATION_GUIDE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/COMPETITION_PRESENTATION_GUIDE.md)	10-Minute pitch video guide, slide structure, and judge Q&A defense playbook.	Pitch Team
🚀 [`SETUP_GUIDE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/SETUP_GUIDE.md)	Local and cloud deployment instructions (Docker, Node.js 20, Go 1.22+, Makefile automation).	Developers & SysAdmins
🌐 [`API_REFERENCE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/API_REFERENCE.md)	REST API endpoint specification with request/response schemas, role authorization, and `/agents/*` multi-agent routes.	Frontend/App Developers
🎨 [`IMAGE_GENERATION_PROMPTS.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/IMAGE_GENERATION_PROMPTS.md)	Curated AI image generation prompts for logos, Bangladesh health cards, UI mockups & posters.	Media & Design Team

Chapter 1: System Architecture Specification

Project Name: MedraLink

Competition: Blockchain Olympiad Bangladesh (BCOLBD 2026)

Track: Student Category — Blockchain / Agentic AI Track

Master Agent Manual: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)

AI Architecture Reference: [`AGENTIC_AI_ARCHITECTURE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/AGENTIC_AI_ARCHITECTURE.md)

Architecture Classification: Multi-Tier Hybrid Permissioned Blockchain & Encrypted Off-Chain Clinical Data Interoperability Network

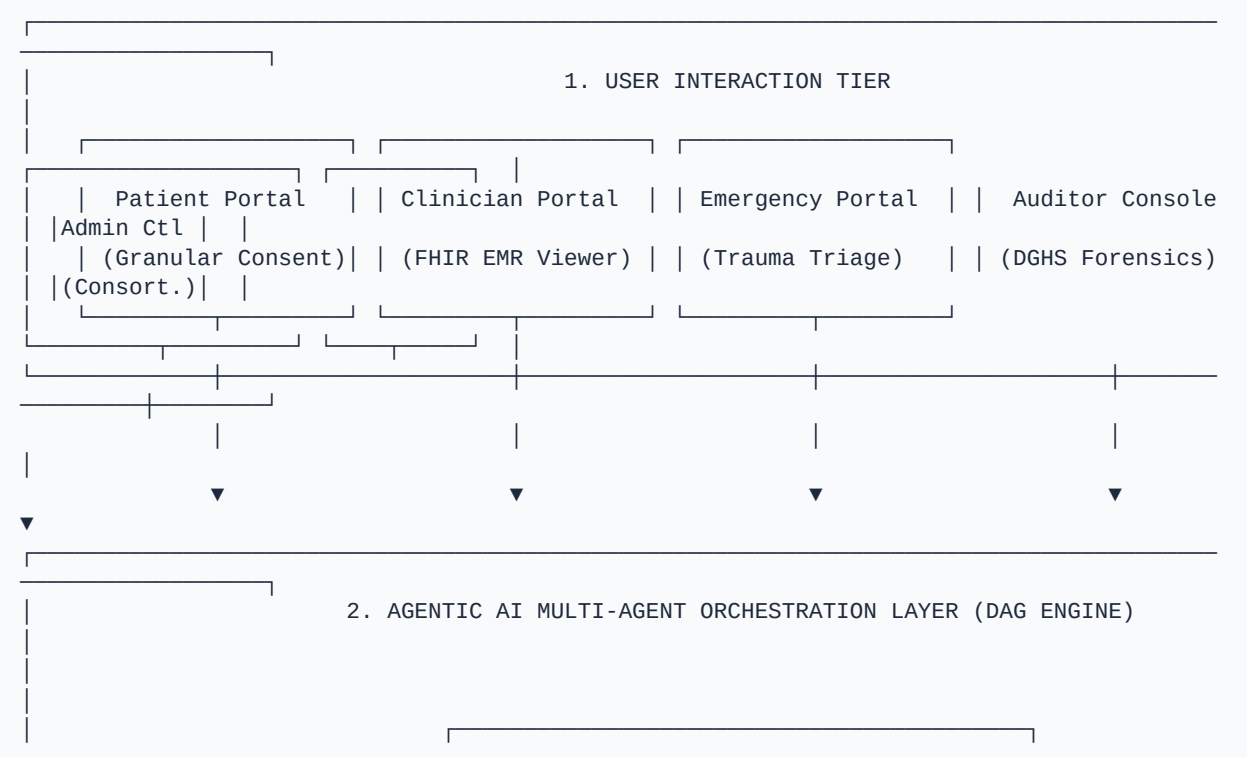
Reference Document: Whitepaper Section D & Figure 1

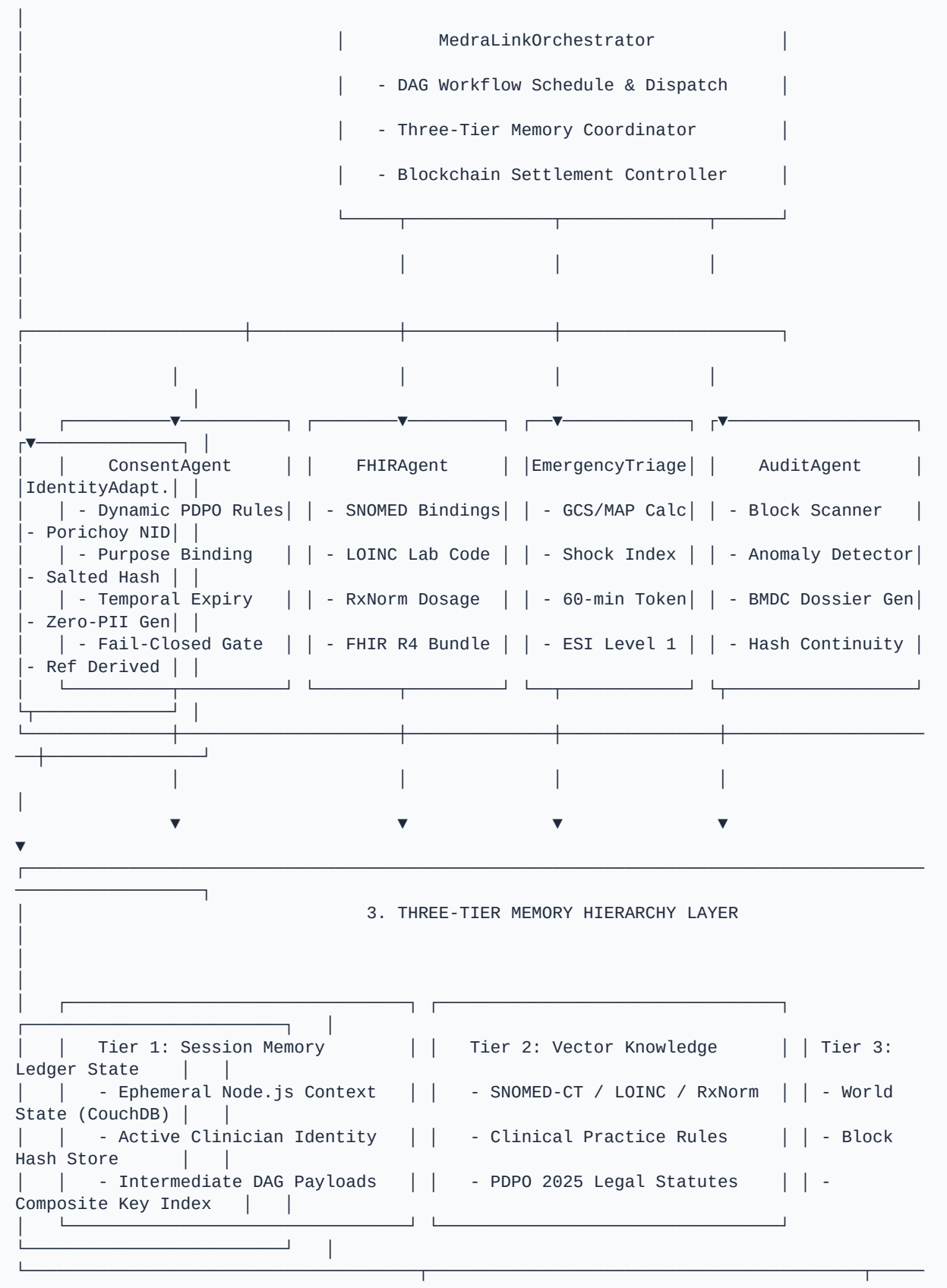
1. Executive Summary

MedraLink is an enterprise-grade healthcare data interoperability and audit provenance system designed to overcome record fragmentation across the pluralistic healthcare ecosystem of Bangladesh (DGHS public facilities and ~5,000 private clinics/diagnostic centers).

The platform separates **data trust, consent enforcement, and audit non-repudiation** (handled on-chain via **Hyperledger Fabric 2.5**) from **heavy clinical payload storage** (handled off-chain via **AES-256-GCM encrypted HL7 FHIR R4 repositories**), fully orchestrated by an **Agentic AI Multi-Agent Directed Acyclic Graph (DAG)**.

2. 🔬 System Topology X-Ray Graph




MedraLinkOrchestrator
- DAG Workflow Schedule & Dispatch
- Three-Tier Memory Coordinator
- Blockchain Settlement Controller
ConsentAgent
- Dynamic PDPO Rules
- Purpose Binding
- Temporal Expiry
- Fail-Closed Gate
FHIRAgent
- SNOMED Bindings
- LOINC Lab Code
- RxNorm Dosage
- FHIR R4 Bundle
EmergencyTriage
- GCS/MAP Calc
- Shock Index
- 60-min Token
- ESI Level 1
AuditAgent
- Block Scanner
- Anomaly Detector
- BMDC Dossier Gen
- Hash Continuity
IdentityAdapt.
- Porichoy NID
- Salted Hash
- Zero-PII Gen
- Ref Derived
3. THREE-TIER MEMORY HIERARCHY LAYER
Tier 1: Session Memory
- Ephemeral Node.js Context
- Active Clinician Identity
- Intermediate DAG Payloads
Tier 2: Vector Knowledge
- SNOMED-CT / LOINC / RxNorm
- Clinical Practice Rules
- PDPO 2025 Legal Statutes
Tier 3: Ledger State
- World State (CouchDB)
- Block Hash Store
- Composite Key Index

```
└──────────────────┘
                                        | gRPC / REST                                  |
Encrypted I/O
                                        ▼                                              ▼
┌──────────────────────────────────────────────────────────────────────┐
┌──────────────────────────────────────────────────┐
│     4. PERMISSIONED BLOCKCHAIN (Hyperledger Fabric 2.5)   | |  5. OFF-CHAIN ENCRYPTED
CLINICAL STORE    |
│   - Channel: `medralink-main`                             | |   - Custodial Hospital
Cloud Vault        |
│   - 9 Canonical Smart Contract Transactions (Go)          | |   - AES-256-GCM Envelope
Encryption (DEK) |
│   - 3-Node Raft Crash Fault Tolerant Consensus Cluster    | |   - Plaintext Zero-PII
FHIR Payloads      |
│   - Cryptographic Proof Anchors: recordHash, patientHash  | |   - Storage Pointer:
s3://vault/rec.enc   |
└──────────────────────────────────────────────────────────────────────┘
└──────────────────────────────────────────────────┘
```

3. 🔐 On-Chain vs. Off-Chain Cryptographic Pipeline X-Ray

```
[ CLINICAL PAYLOAD / UNSTRUCTURED NOTE ]
                    |
                    ▼ (FHIRAgent Semantic Normalization)
     [ HL7 FHIR R4 Bundle (JSON) ]
                    |
                    ▼ (AES-256-GCM Envelope Encryption with Unique DEK)
                    ┴
    ┌──────────────────────────────┐
    |                              |
    ▼                              ▼
[ Ciphertext Payload ]      [ recordHash = SHA256(Ciphertext) ]
    |                       [ opaquePointerHash = SHA256(s3://...) ]
    ▼                              |
(Stored Off-Chain in               ▼ (CreateRecordReference)
 Custodial Hospital Vault)   [ Anchored on Hyperledger Fabric Ledger ]
```

Dimension	On-Chain (Hyperledger Fabric Ledger)	Off-Chain (Custodial Repositories)
Identity Data	Pseudonymous `patientRefHash = SALTED_SHA256(HealthId + DOB)`	Demographics, Patient Name, Phone, Address (Encrypted)
National ID (NID)	**NEVER STORED** (Discarded by adapter after hash derivation)	Stored only in external government NID registry
Clinical Records	Cryptographic integrity `recordHash = SHA256(ciphertext)` + pointer hash	Plaintext FHIR R4 Resource Bundles encrypted with AES-256-GCM

Consent Model	Granular scope tokens, purpose allowlist, expiration timestamp, status	Detailed patient-signed legal agreement forms
Audit Trails	Complete immutable sequence of every access attempt & emergency event	System debug telemetry and access analytics
Right to Erasure	Cryptographic hash retained as non-repudiable tombstone	Ciphertext and encryption keys permanently deleted

4. ⚡ Transaction Execution & Raft Consensus Pipeline

```
sequenceDiagram
    autonumber
    actor Client as Clinician / Patient Portal
    participant GW as API Gateway (Node.js)
    participant Agents as Agentic AI DAG Engine
    participant Peer as Fabric Peer (Org1 / Org2)
    participant Orderer as 3-Node Raft Consensus Cluster
    participant Ledger as Fabric Ledger (CouchDB + Blockstore)

    Client->>GW: Submit Transaction Request
    GW->>Agents: Evaluate Policy / Normalize Payload
    Agents-->>GW: Verified Intent & Normalized Parameters
    GW->>Peer: Endorsement Proposal (Invoke Chaincode)
    Peer->>Peer: Execute Go Chaincode & Simulate Read/Write Sets
    Peer-->>GW: Proposal Response with Cryptographic Signature
    GW->>Orderer: Broadcast Signed Transaction Envelope
    Orderer->>Orderer: Raft Consensus (Leader Proposes Block #N)
    Orderer->>Peer: Deliver Ordered Block to Consortium Peers
    Peer->>Ledger: Validate MVCC & Commit Block to World State
    Peer-->>GW: Emit Chaincode Event (e.g. ConsentGranted, AccessLogged)
    GW-->>Client: Transaction Receipt (txId, blockNumber, status: VALID)
```

5. 🏛️ Consortium Organizations & Node Topologies

The pilot deployment configures a **3+1 Consortium** topology:

5.1 Org1MSP — Pilot Hospital A (Tertiary Public / Academic Facility)

- **Nodes:** 1 Peer (`peer0.org1`), 1 CA (`ca.org1`), 1 Raft Orderer (`orderer1`).
- **State Database:** CouchDB instance `couchdb0` (Port 5984).
- **Role:** Endorses patient registration, consent grants, and records generated at Hospital A.

5.2 Org2MSP — Pilot Hospital B (Private Hospital & Diagnostic Center)

- **Nodes:** 1 Peer (`peer0.org2`), 1 CA (`ca.org2`), 1 Raft Orderer (`orderer2`).
- **State Database:** CouchDB instance `couchdb1` (Port 6984).
- **Role:** Endorses lab diagnostic reports, emergency department break-glass events, and external access requests.

5.3 Org3MSP — Medralink Consortium Operator

- **Nodes:** 1 Shadow Peer (`peer0.org3`), 1 CA (`ca.org3`), 1 Raft Orderer (`orderer3`).
- **State Database:** CouchDB instance `couchdb2` (Port 7984).
- **Role:** Maintains gateway routing, ordering service orchestration, and network health monitoring. Does not endorse patient consent.

5.4 OrgAuditorMSP — Regulatory Observer (DGHS / MIS Bangladesh)

- **Nodes:** 1 Read-Only Audit Peer (`peer0.orgauditor`), 1 CA (`ca.orgauditor`).
- **State Database:** CouchDB instance `couchdb3` (Port 8984).
- **Role:** Non-endorsing audit replica. Receives all ledger blocks via gossip and participates in `ReviewEmergencyAccess` workflows.

6. 🔗 Reference Connections

- Master Agent Architecture & Guidelines: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)
- Agentic AI Architecture: [`AGENTIC_AI_ARCHITECTURE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/AGENTIC_AI_ARCHITECTURE.md)
- Smart Contract Technical Specification: [`CHAINCODE_SPECIFICATION.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/CHAINCODE_SPECIFICATION.md)
- FHIR Interoperability Guide: [`FHIR_INTEROPERABILITY_GUIDE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/FHIR_INTEROPERABILITY_GUIDE.md)
- Governance & Security Manual: [`GOVERNANCE_AND_SECURITY_MANUAL.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/GOVERNANCE_AND_SECURITY_MANUAL.md)
- Prototype Demo Script: [`DEMO_SCRIPT.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/DEMO_SCRIPT.md)

Chapter 2: Autonomous Agentic AI Multi-Agent Architecture

Project Name: MedraLink

Competition: Blockchain Olympiad Bangladesh (BCOLBD 2026)

Track: Student Category — Blockchain / Agentic AI Track

Master Agent Manual: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)

Architecture Classification: Autonomous Multi-Agent Directed Acyclic Graph (DAG) with 3-Tier Memory Hierarchy

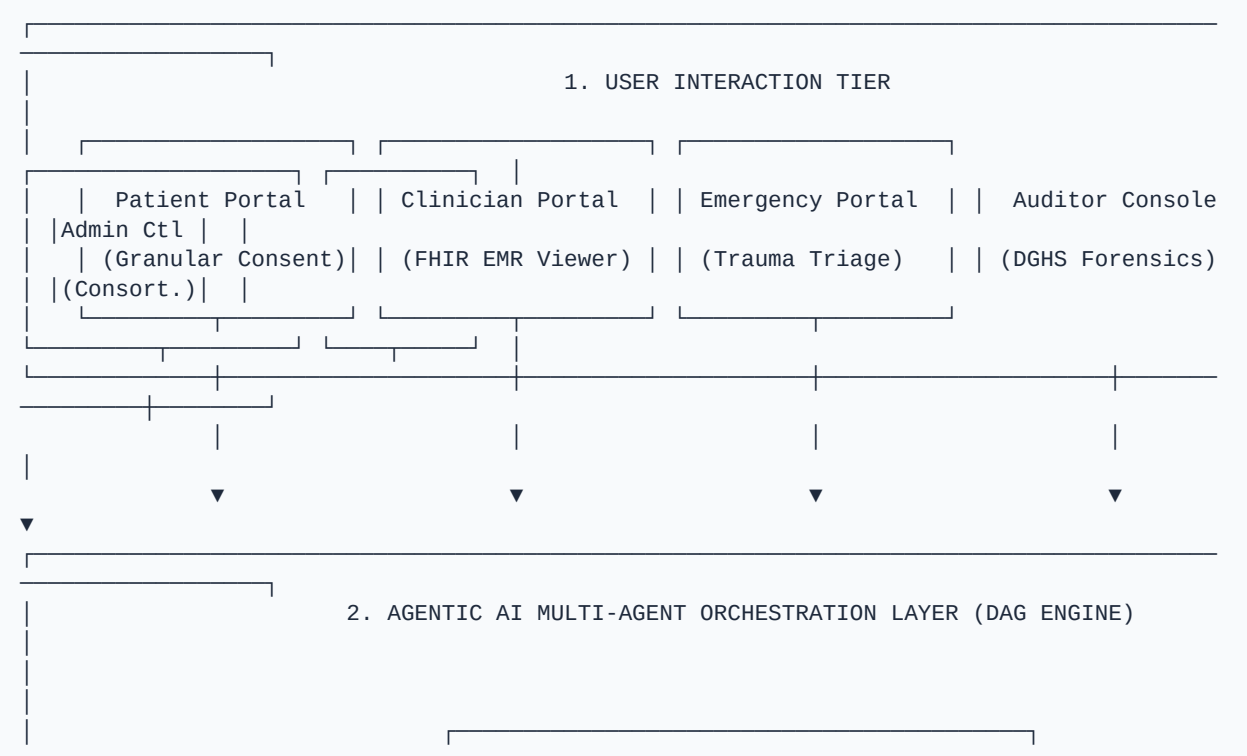
Reference Document: Master Whitepaper Section D & Figure 1

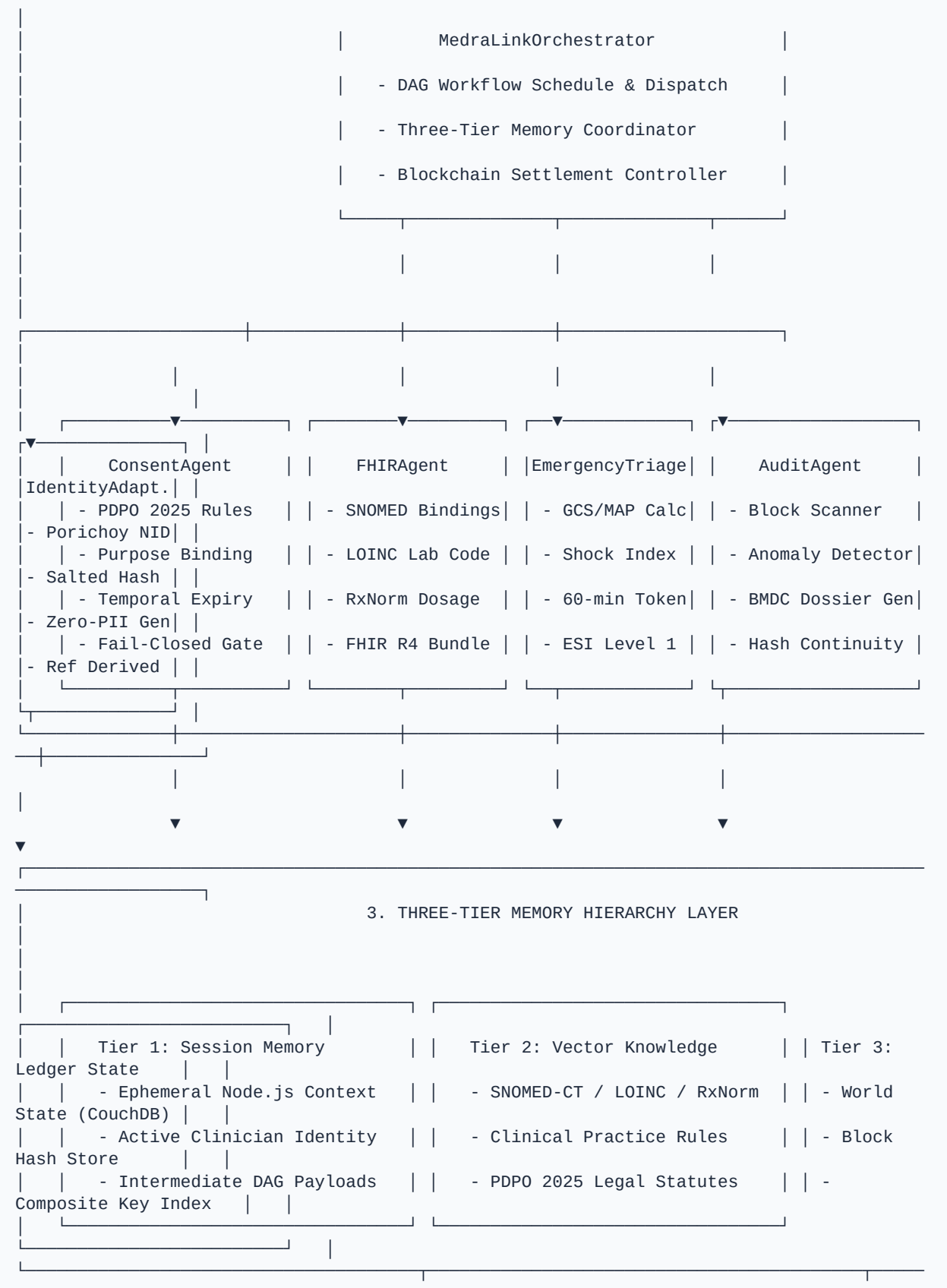
1. Executive Summary & Agentic Mission

MedraLink deploys an **Autonomous Agentic AI Multi-Agent Orchestration Layer** uniting **five specialized autonomous AI agents** operating in an orchestrated Directed Acyclic Graph (DAG). The agents bridge the gap between messy, unstructured clinical workflows across Bangladesh's ~5,000 healthcare facilities and the strict, immutable guarantees of a **Hyperledger Fabric 2.5** permissioned blockchain consortium.

Every agent is purpose-built, deterministic in security-critical invariants (such as Zero-PII assertions, fail-closed policy enforcement, and 60-minute time-boxed emergency limits), and operates across a **Three-Tier Memory Hierarchy**.

2. 🔬 System X-Ray Architecture Graph




MedraLinkOrchestrator
- DAG Workflow Schedule & Dispatch
- Three-Tier Memory Coordinator
- Blockchain Settlement Controller
ConsentAgent
- PDPO 2025 Rules
- Purpose Binding
- Temporal Expiry
- Fail-Closed Gate
FHIRAgent
- SNOMED Bindings
- LOINC Lab Code
- RxNorm Dosage
- FHIR R4 Bundle
EmergencyTriage
- GCS/MAP Calc
- Shock Index
- 60-min Token
- ESI Level 1
AuditAgent
- Block Scanner
- Anomaly Detector
- BMDC Dossier Gen
- Hash Continuity
IdentityAdapt.
- Porichoy NID
- Salted Hash
- Zero-PII Gen
- Ref Derived
3. THREE-TIER MEMORY HIERARCHY LAYER
Tier 1: Session Memory
- Ephemeral Node.js Context
- Active Clinician Identity
- Intermediate DAG Payloads
Tier 2: Vector Knowledge
- SNOMED-CT / LOINC / RxNorm
- Clinical Practice Rules
- PDPO 2025 Legal Statutes
Tier 3: Ledger State
- World State (CouchDB)
- Block Hash Store
- Composite Key Index

```
└──────────────────┘
                                        | gRPC / REST                                  |
Encrypted I/O
                                        ▼                                              ▼
┌──────────────────────────────────────────────────────────────────┐
┌──────────────────────────────────────────────┐
│     4. PERMISSIONED BLOCKCHAIN (Hyperledger Fabric 2.5)   | |  5. OFF-CHAIN ENCRYPTED
CLINICAL STORE    |
|   - Channel: `medralink-main`                             | |   - Custodial Hospital
Cloud Vault        |
|   - 9 Canonical Smart Contract Transactions (Go)          | |   - AES-256-GCM Envelope
Encryption (DEK) |
|   - 3-Node Raft Crash Fault Tolerant Consensus Cluster    | |   - Plaintext Zero-PII
FHIR Payloads      |
|   - Cryptographic Proof Anchors: recordHash, patientHash  | |   - Storage Pointer:
s3://vault/rec.enc   |
└──────────────────────────────────────────────────────────────────┘
└──────────────────────────────────────────────┘
```

3. 🔄 Multi-Agent Directed Acyclic Graph (DAG) Execution Flow

```
graph TD
    classDef orchestrator fill:#1E1B4B,stroke:#6366F1,stroke-width:2px,color:#FFFFFF;
    classDef agent fill:#0F172A,stroke:#38BDF8,stroke-width:2px,color:#FFFFFF;
    classDef critical fill:#450A0A,stroke:#EF4444,stroke-width:2px,color:#FFFFFF;
    classDef ledger fill:#064E3B,stroke:#10B981,stroke-width:2px,color:#FFFFFF;
    classDef memory fill:#312E81,stroke:#818CF8,stroke-width:1px,color:#FFFFFF;

    Client[Clinician / Emergency / Patient Portal] -->|Task Payload|
Orch[MedraLinkOrchestrator DAG Planner]:::orchestrator

    subgraph MultiAgentDAG [Autonomous Agent DAG Execution]
        Orch -->|Route Clinical Intake| FHIR[FHIRAgent Normalizer]:::agent
        Orch -->|Route Access Query| Consent[ConsentAgent Policy Guard]:::agent
        Orch -->|Route Trauma Incident| EMG[EmergencyTriageAgent]:::critical
        Orch -->|Route Scheduled / Forensic Scan| Audit[AuditAgent Forensic
Scanner]:::agent

        FHIR -->|SNOMED / LOINC / RxNorm Extracted| Mem2[(Tier 2: Knowledge Vector
Index)]:::memory
        FHIR -->|Normalized FHIR R4 Bundle| Consent

        EMG -->|GCS <= 8 / Shock Index > 1.0| LifeSafety[PDPO 2025 Sec 24 Life-Safety
Override]:::critical
        LifeSafety -->|60-min Break-Glass Token| Consent

        Consent -->|Dynamic Policy Evaluation| DecDecision{Policy Verdict}
        DecDecision -->|ALLOW: Token Valid & Scope Bound| Settle[Blockchain Settlement
Engine]:::ledger
        DecDecision -->|DENY: Expired / Revoked / No Consent| FailClosed[Fail-Closed
Access Rejection]:::critical

        Audit -->|Scan Block Sequence & Verify SHA-256 Chains| AuditReport[DGHS Forensic
Dossier Generator]:::agent
    end
```

```
    Settle -->|Canonical Tx Execution| Fabric[(Tier 3: Hyperledger Fabric 2.5
Ledger)]:::ledger
    Settle -->|AES-256-GCM Envelope Encryption| OffChainStore[(Off-Chain Custodial
Repository)]:::ledger
    FailClosed -->|Log Access Denied Event| Fabric
```

4. 🤖 Autonomous Agents Deep-Dive

4.1 `ConsentAgent` — Dynamic Policy Guard

- **Governance Standard:** Bangladesh Personal Data Protection Ordinance (**PDPO 2025**) & GDPR Article 9.
- **Verification Pipeline (Fail-Closed Execution):**

1. *Emergency Bypass Check:* If valid break-glass token exists, approve access under Section 24 life-safety exemption.
2. *Active Consent Token Query:* Verify presence of `CONSENT_<id>` in Tier 3 Ledger.
3. *Revocation Invariant:* If `revoked == true`, immediately terminate evaluation with `DENIED_REVOKED_CONSENT`.
4. *Temporal Limit Check:* Assert $\text{Unix}(\text{now}) \le \text{expiryTimestamp}$.
5. *Scope & Purpose Allowlist:* Assert $\text{requestedScope} \subseteq \text{grantedScope}$ and $\text{purpose} == \text{consentedPurpose}$.

4.2 `FHIRAgent` — Semantic Normalization Engine

- **Mission:** Ingests unformatted clinical notes and maps them to international ontologies:
- **SNOMED-CT:** Diagnoses and adverse reactions (`373270004` Penicillin Allergy, `39579001` Anaphylaxis, `44054006` Type 2 Diabetes).
- **LOINC:** Laboratory diagnostic observations (`1558-6` Fasting Glucose, `4548-4` HbA1c, `8867-4` Heart Rate).
- **RxNorm:** Pharmaceutical ingredients and formulations (`860975` Metformin 500mg, `312961` Amlodipine 5mg).
- **Output:** Validated `Bundle` collection for off-chain AES-256-GCM encryption.

4.3 `EmergencyTriageAgent` — Trauma Resuscitation Engine

- **Triage Algorithms:**
- $\text{GCS} \le 8$: Severe coma / traumatic brain injury alert.
- $\text{MAP} = \frac{2 \times \text{DBP} + \text{SBP}}{3} < 65\text{ mmHg}$: Hypotensive shock alert.
- $\text{Shock Index} = \frac{\text{Heart Rate}}{\text{Systolic BP}} > 1.0$: Imminent circulatory collapse.
- **Emergency Severity Index (ESI):** Automatically assigned ESI Level 1 (Resuscitation).
- **Break-Glass Token:** Dispenses a cryptographic token valid for **exactly 60 minutes**, emitting a high-priority audit notification to DGHS.

4.4 `AuditAgent` — Forensic Block Scanner

- **Forensic Vectors:**

6. *Unreviewed Break-Glass Audit:* Flags all `InvokeEmergencyAccess` events lacking a corresponding `ReviewEmergencyAccess` transaction.
7. *Denial Burst Detection:* Identifies abnormal patterns of rejected requests.

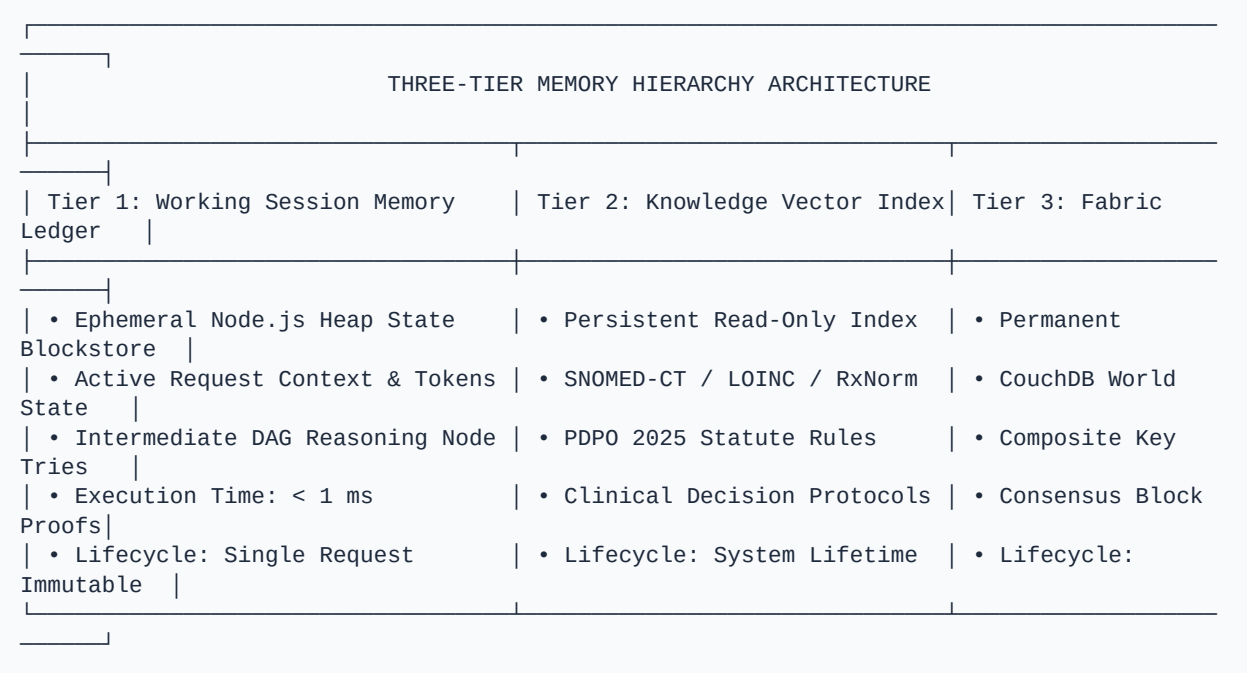
8. *Cryptographic Hash Chain Continuity:* Asserts $\forall i > 0, \text{Block}[i].\text{previousHash} == \text{Block}[i-1].\text{dataHash}$.

- **Output:** Cryptographic evidence dossier (`SHA256(Findings)`) formatted for the DGHS regulatory inspector.

4.5 `MedraLinkOrchestrator` — Master DAG Planner

- **Mission:** Manages inter-agent message buses, maintains intermediate context in Tier 1 Working Session Memory, retrieves ontology representations from Tier 2 Vector Knowledge Index, and commits immutable transactions to Tier 3 Hyperledger Fabric Ledger.

5. 🧠 Three-Tier Memory Hierarchy X-Ray



6. 🔗 Reference Connections

- Master Agent Architecture & Guidelines: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)
- System Topology Specification: [`ARCHITECTURE_SPECIFICATION.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/ARCHITECTURE_SPECIFICATION.md)
- Canonical Smart Contract Specification: [`CHAINCODE_SPECIFICATION.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/CHAINCODE_SPECIFICATION.md)
- HL7 FHIR Interoperability Guide: [`FHIR_INTEROPERABILITY_GUIDE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/FHIR_INTEROPERABILITY_GUIDE.md)

- Governance and Security Manual: [`GOVERNANCE_AND_SECURITY_MANUAL.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/GOVERNANCE_AND_SECURITY_MANUAL.md)
- Step-by-Step Demo Script: [`DEMO_SCRIPT.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/DEMO_SCRIPT.md)

Chapter 3: Hyperledger Fabric Smart Contract Specification

Contract Package: `medralink-cc`

Language: Go (v1.22+)

Framework: `github.com/hyperledger/fabric-contract-api-go/v2`

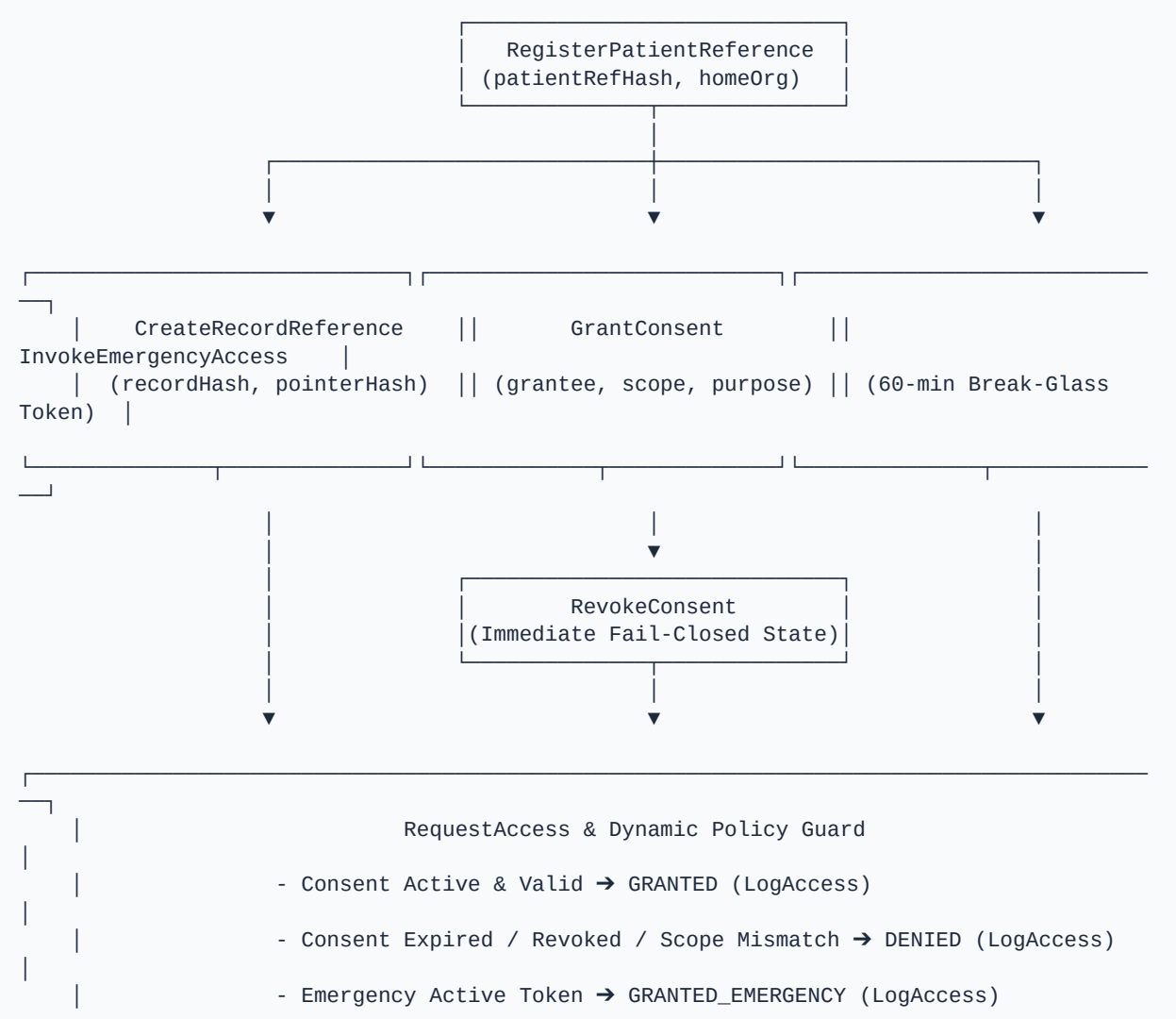
Channel: `medralink-main`

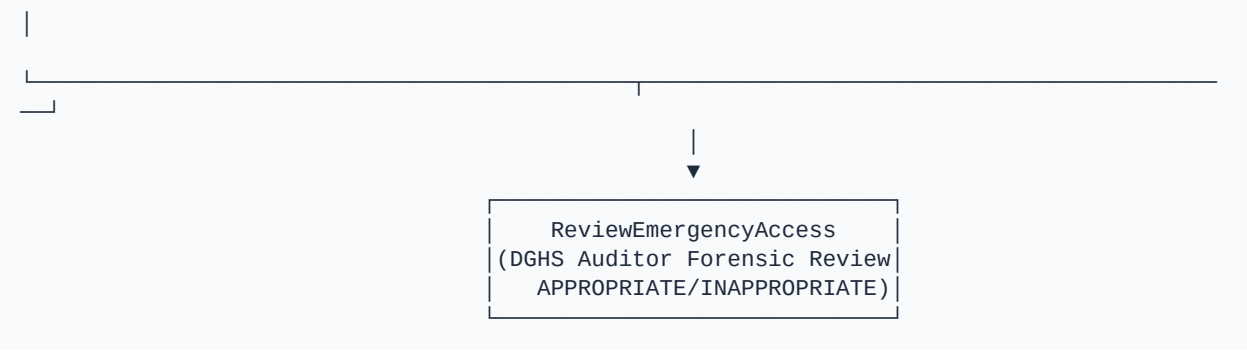
Master Agent Manual: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)

AI Architecture Reference: [`AGENTIC_AI_ARCHITECTURE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/AGENTIC_AI_ARCHITECTURE.md)

Standard: 9 Canonical Transactions (Whitepaper Table 9 & Section G3)

1. 🔬 Chaincode State Machine & Transaction X-Ray Graph





2. 🌳 Composite Key Trie & State Indexing X-Ray

```
WORLD STATE STORE (CouchDB / LevelDB)
├── <patientRefHash> ──────────────────────➔ PatientReference Object
├── PROV_<providerIdHash> ─────────────────➔ ProviderReference Object
├── REC_<recordId> ────────────────────────➔ RecordReference Object
├── CONSENT_<consentId> ───────────────────➔ Consent Object
├── AUDIT_<requestId> ─────────────────────➔ AccessEvent Object
├── EMERGENCY_<emergencyId> ───────────────➔ EmergencyAccessEvent Object
│
└── SECONDARY COMPOSITE KEY TRIES:
    ├── patient~record
    │   └── \x00patient~record\x00<patientRefHash>\x00<recordId>\x00 ➔ \x00
    │
    ├── patient~consent
    │   └── \x00patient~consent\x00<patientRefHash>\x00<consentId>\x00 ➔ \x00
    │
    ├── patient~audit
    │   └── \x00patient~audit\x00<patientRefHash>\x00<requestId>\x00 ➔ \x00
    │
    └── patient~emergency
        └── \x00patient~emergency\x00<patientRefHash>\x00<emergencyId>\x00 ➔ \x00
```

3. Canonical Transaction Signatures

#	Transaction Name	Primary Actor	Invocation Path	Event Emitted
1	`RegisterPatientReference`	Identity Adapter / Gateway	`POST /patients/register`	`PatientRegistered`
2	`RegisterProvider`	Hospital Administrator	`POST /providers/register`	`ProviderRegistered`
3	`CreateRecordReference`	Authorized Clinician	`POST /records`	`RecordCreated`
4	`GrantConsent`	Patient App	`POST /consents`	`ConsentGranted`
5	`RevokeConsent`	Patient App	`DELETE	`ConsentRevoked`

			/consents/:id`	
6	`RequestAccess`	Authorized Clinician	`POST /access/request`	`AccessRequested`
7	`LogAccess`	API Gateway	Auto-invoked upon access attempt	`AccessLogged`
8	`InvokeEmergencyAccess`	Emergency Clinician	`POST /emergency/invoke`	`EmergencyAccessInvoked`
9	`ReviewEmergencyAccess`	Network Auditor	`POST /emergency/review`	`EmergencyAccessReviewed`

4. Detailed Transaction Specifications

4.1 `RegisterPatientReference`

- **Signature:** `RegisterPatientReference(ctx, patientRefHash string, homeOrg string, createdAt string) (*PatientReference, error)`
- **Caller Requirement:** Gateway Certificate + Admin Attestation (`OU=Admin` or `OU=Client`).
- **Validation Rules:**

9. `patientRefHash` must not already exist in World State.
10. `AssertZeroPII`: Input cannot contain raw 10-, 13-, or 17-digit national identity numbers.

- **State Writes:** Key `<patientRefHash>`, Value: `PatientReference` JSON.
- **Event:** `PatientRegistered`.

4.2 `RegisterProvider`

- **Signature:** `RegisterProvider(ctx, providerIDHash string, org string, role string, certSerial string, createdAt string) (*ProviderReference, error)`
- **Caller Requirement:** `OU=Admin` of the target MSP.
- **Validation Rules:** Valid SHA-256 hash, permitted roles (`Clinician`, `Emergency`, `Admin`, `Auditor`).
- **State Writes:** Key `PROV_<providerIDHash>`, Value: `ProviderReference` JSON.
- **Event:** `ProviderRegistered`.

4.3 `CreateRecordReference`

- **Signature:** `CreateRecordReference(ctx, recordID string, patientRefHash string, recordType string, recordHash string, opaquePointerHash string, custodialOrg string, provenance string, createdAt string) (*RecordReference, error)`
- **Caller Requirement:** `OU=Clinician` + Endorsement from custodial hospital organization.
- **Validation Rules:** Verified patient existence, valid FHIR scope resource type, Zero-PII assertions.
- **State Writes:** Key `REC_<recordID>` and Composite Key `patient~record`.
- **Event:** `RecordCreated`.

4.4 `GrantConsent`

- **Signature:** `GrantConsent(ctx, consentID string, patientRefHash string, grantee string, scopeJSON string, purpose string, expiryTimestamp string, patientSig string, createdAt string) (*Consent, error)`

- **Caller Requirement:** Patient App digital signature.
- **Validation Rules:** Granular scopes (no wildcards `*`), valid ISO8601/RFC3339 timestamp, approved purpose code.
- **State Writes:** Key `CONSENT_<consentID>` and Composite Key `patient~consent`.
- **Event:** `ConsentGranted`.

4.5 `RevokeConsent`

- **Signature:** `RevokeConsent(ctx, consentID string, patientRefHash string, patientSig string) (*Consent, error)`
- **Caller Requirement:** Patient reference match.
- **Validation Rules:** Asserts consent exists, matches patient, and is not already revoked.
- **State Writes:** Sets `revoked = true` under `CONSENT_<consentID>`.
- **Event:** `ConsentRevoked`.

4.6 `RequestAccess`

- **Signature:** `RequestAccess(ctx, requestID string, patientRefHash string, consentID string, accessorHash string, scope string, purpose string) (*AccessVerificationResult, error)`
- **Caller Requirement:** Authorized clinician credentials.
- **Validation Rules:** Dynamic verification of patient ownership, revocation flag, expiration time, scope allowlist, and purpose alignment.
- **Event:** `AccessRequested`.

4.7 `LogAccess`

- **Signature:** `LogAccess(ctx, requestID string, patientRefHash string, accessorHash string, scope string, purpose string, status string, timestamp string) (*AccessEvent, error)`
- **State Writes:** Key `AUDIT_<requestID>` and Composite Key `patient~audit`.
- **Event:** `AccessLogged`.

4.8 `InvokeEmergencyAccess`

- **Signature:** `InvokeEmergencyAccess(ctx, emergencyID string, clinicianIDHash string, patientRefHash string, reasonCode string, scopeJSON string, expiryTimestamp string, createdAt string) (*EmergencyAccessEvent, error)`
- **Caller Requirement:** Emergency department physician credentials (`OU=Emergency`).
- **Validation Rules:** Valid emergency reason code, granular scopes, valid expiration timestamp.
- **State Writes:** Key `EMERGENCY_<emergencyID>`, Composite Key `patient~emergency`, and audit entry `AUDIT_EMG_<emergencyID>`.
- **Event:** `EmergencyAccessInvoked`.

4.9 `ReviewEmergencyAccess`

- **Signature:** `ReviewEmergencyAccess(ctx, emergencyID string, auditorIDHash string, reviewStatus string, findingsHash string, reviewedAt string) (*EmergencyAccessEvent, error)`
- **Caller Requirement:** Licensed compliance auditor (`OU=Auditor`, `OrgAuditorMSP`).
- **Validation Rules:** Review status must be `APPROPRIATE` or `INAPPROPRIATE`.
- **State Writes:** Updates review state under `EMERGENCY_<emergencyID>`.
- **Event:** `EmergencyAccessReviewed`.

5. Query Methods

- `GetPatientReference(ctx, patientRefHash)`
- `GetRecordReference(ctx, recordID)`
- `GetConsent(ctx, consentID)`
- `GetEmergencyEvent(ctx, emergencyID)`
- `GetAccessHistory(ctx, patientRefHash)`: Uses `patient~audit` composite index.
- `GetRecordsForPatient(ctx, patientRefHash)`: Uses `patient~record` composite index.
- `GetConsentsForPatient(ctx, patientRefHash)`: Uses `patient~consent` composite index.
- `GetEmergencyEventsForPatient(ctx, patientRefHash)`: Uses `patient~emergency` composite index.

6. 🔗 Reference Connections

- Master Agent Architecture & Guidelines: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)
- Agentic AI Architecture: [`AGENTIC_AI_ARCHITECTURE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/AGENTIC_AI_ARCHITECTURE.md)
- System Architecture Specification: [`ARCHITECTURE_SPECIFICATION.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/ARCHITECTURE_SPECIFICATION.md)
- Testing and QA Report: [`TESTING_AND_QA_REPORT.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/TESTING_AND_QA_REPORT.md)

Chapter 4: HL7 FHIR R4 Interoperability & Semantic Normalization

Standard Version: HL7 FHIR Release 4 (R4)

Encryption Standard: AES-256-GCM (Authenticated Encryption with Associated Data)

Terminology Standard: SNOMED-CT, LOINC, RxNorm, ICD-10

National Integration Target: DGHS Shared Health Record (SHR) & OpenMRS+

Master Agent Manual: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)

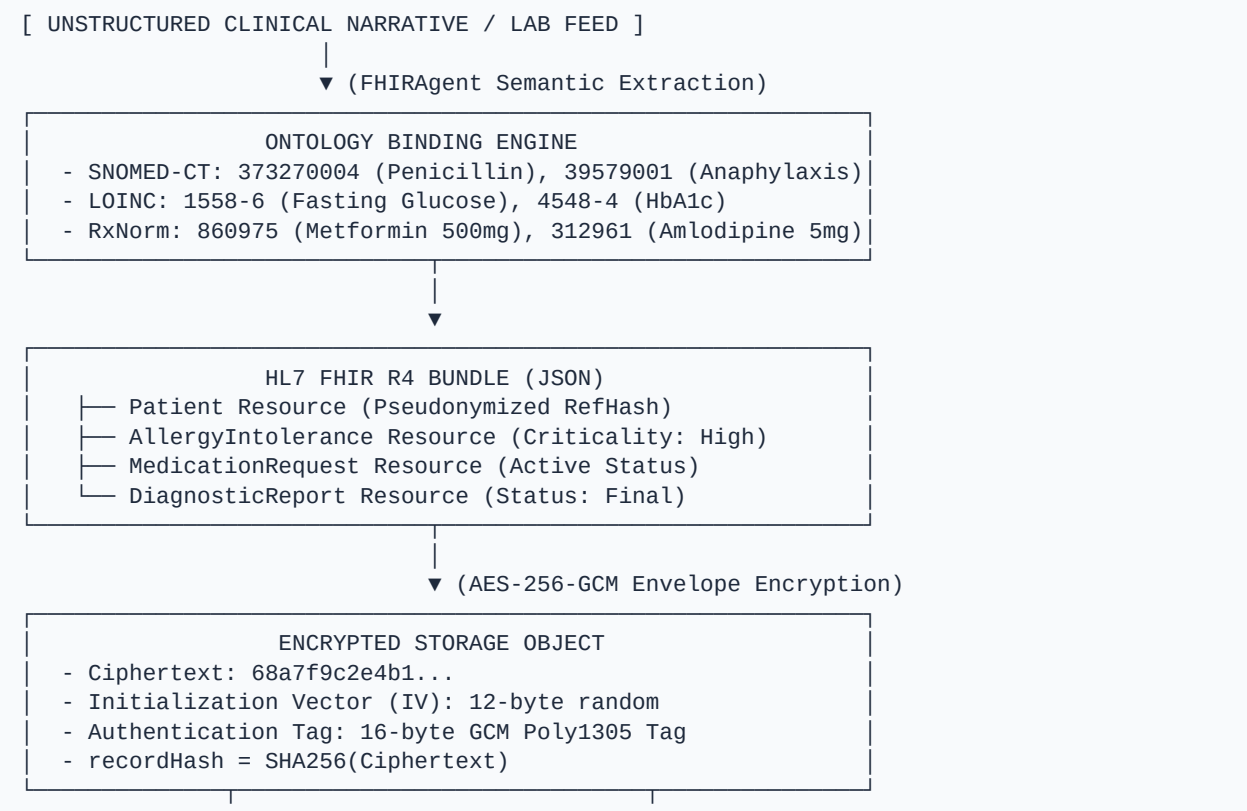
AI Architecture Reference: [`AGENTIC_AI_ARCHITECTURE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/AGENTIC_AI_ARCHITECTURE.md)

1. Overview & Semantic Interoperability Philosophy

MedraLink is designed to be **SHR-compatible, not SHR-competitive**. Rather than replacing existing hospital Electronic Medical Record (EMR) databases or the DGHS Shared Health Record exchange, MedraLink serves as the **decentralized trust, consent, and immutable provenance layer**.

The **`FHIRAgent`** semantic normalization engine ingests unstructured physician notes, prescription forms, and legacy diagnostic outputs, binding them to international medical terminologies and generating standardized **HL7 FHIR R4 Bundles**.

2. 🔬 FHIR Normalization & Cryptographic Envelope X-Ray



```
                 |                                |
                 ▼                                ▼
(Custodial Off-Chain Storage)      (CreateRecordReference)
s3://hospital-vault/rec-01.enc     Hyperledger Fabric Ledger
```

3. Supported FHIR R4 Clinical Resources

3.1 `AllergyIntolerance` (Critical Patient Safety)

- **Use Case:** High-criticality drug and food allergies (e.g. Severe Penicillin Anaphylaxis).
- **Terminology Binding:** SNOMED-CT (e.g. `373270004` — Penicillin structure, `39579001` — Anaphylaxis).
- **Structure:**

```
  {
    "resourceType": "AllergyIntolerance",
    "id": "allergy-001",
    "clinicalStatus": {
      "coding": [{ "system": "http://terminology.hl7.org/CodeSystem/allergyintolerance-
clinical", "code": "active" }]
    },
    "criticality": "high",
    "code": {
      "coding": [{ "system": "http://snomed.info/sct", "code": "373270004", "display":
"Penicillin" }],
      "text": "Penicillin (Severe Anaphylaxis Risk)"
    },
    "reaction": [{
      "manifestation": [{ "coding": [{ "system": "http://snomed.info/sct", "code":
"39579001", "display": "Anaphylaxis" }] }],
      "severity": "severe"
    }]
  }
```

3.2 `MedicationRequest` (Active Pharmacotherapy)

- **Use Case:** Prescriptions and long-term medication management.
- **Terminology Binding:** RxNorm (e.g. `860975` — Metformin hydrochloride 500 MG).
- **Structure:**

```
  {
    "resourceType": "MedicationRequest",
    "id": "med-001",
    "status": "active",
    "intent": "order",
    "medicationCodeableConcept": {
      "coding": [{ "system": "http://www.nlm.nih.gov/research/umls/rxnorm", "code":
"860975", "display": "Metformin 500mg" }],
      "text": "Metformin 500mg daily"
    }
  }
```

3.3 `DiagnosticReport` (Laboratory and Diagnostic Results)

- **Use Case:** Blood chemistry, pathology, radiology summaries.
- **Terminology Binding:** LOINC (e.g. `1558-6` — Fasting Blood Glucose).
- **Structure:**

```
  {
    "resourceType": "DiagnosticReport",
    "id": "diag-001",
    "status": "final",
    "code": {
      "coding": [{ "system": "http://loinc.org", "code": "1558-6", "display": "Fasting
Glucose" }]
    },
    "conclusion": "Elevated fasting blood sugar (7.8 mmol/L). Consistent with mild
diabetes mellitus."
  }
```

4. Encryption & Key Management (AES-256-GCM)

11. **Envelope Encryption:** Each clinical resource bundle is encrypted under an ephemeral 256-bit Data Encryption Key (DEK).
12. **Authenticated Encryption (GCM):** The 128-bit authentication tag detects any bit-level tampering in custodial storage.
13. **Ledger Integrity Anchor:** The SHA-256 hash of the ciphertext (`recordHash`) is committed on-chain. Before decryption, the client asserts:

$$\text{SHA256}(\text{StoredCiphertext}) == \text{OnChainRecordHash}$$

5. 🔗 Reference Connections

- Master Agent Architecture & Guidelines: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)
- Agentic AI Architecture: [`AGENTIC_AI_ARCHITECTURE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/AGENTIC_AI_ARCHITECTURE.md)
- System Architecture Specification: [`ARCHITECTURE_SPECIFICATION.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/ARCHITECTURE_SPECIFICATION.md)
- REST API Reference: [`API_REFERENCE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/API_REFERENCE.md)

Chapter 5: Consortium Governance & Security Manual

Consortium Model: 3-Pillar Federated Healthcare Consortium

Regulatory Framework: Bangladesh Personal Data Protection Ordinance (PDPO 2025) & DGHS Interoperability Guidelines

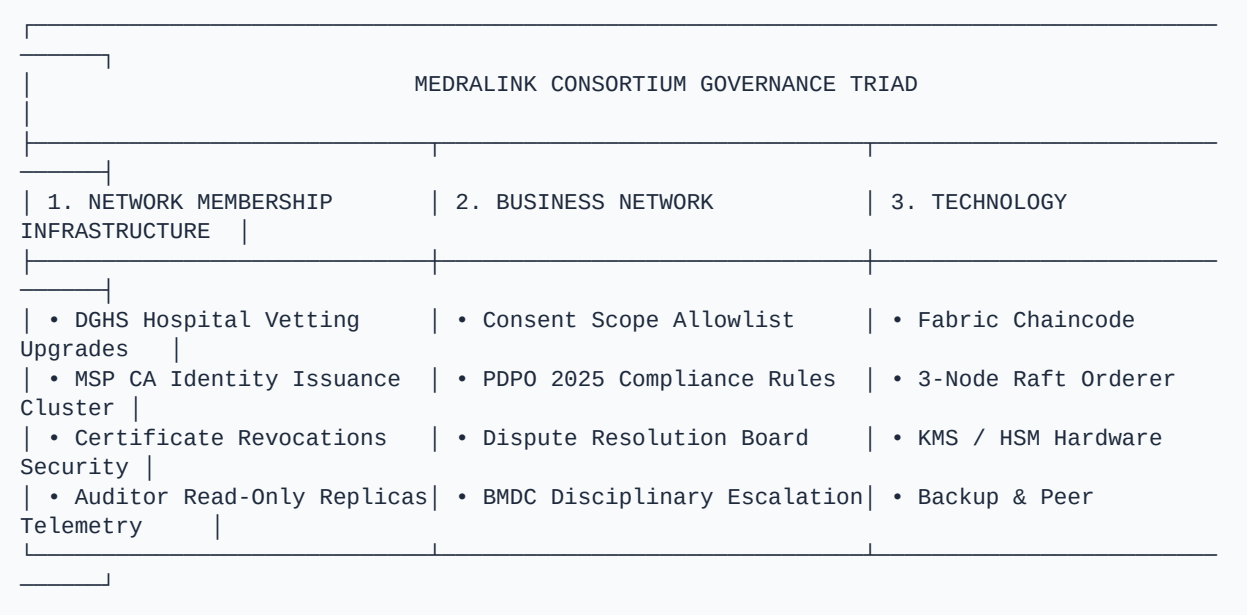
Cryptographic Tokenomics: 100% Tokenless / Zero Cryptocurrency

Master Agent Manual: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)

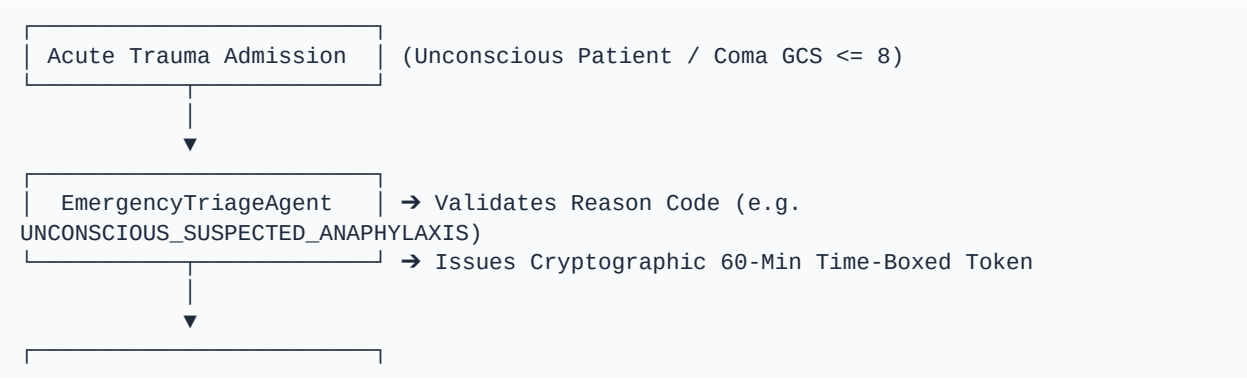
AI Architecture Reference: [`AGENTIC_AI_ARCHITECTURE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/AGENTIC_AI_ARCHITECTURE.md)

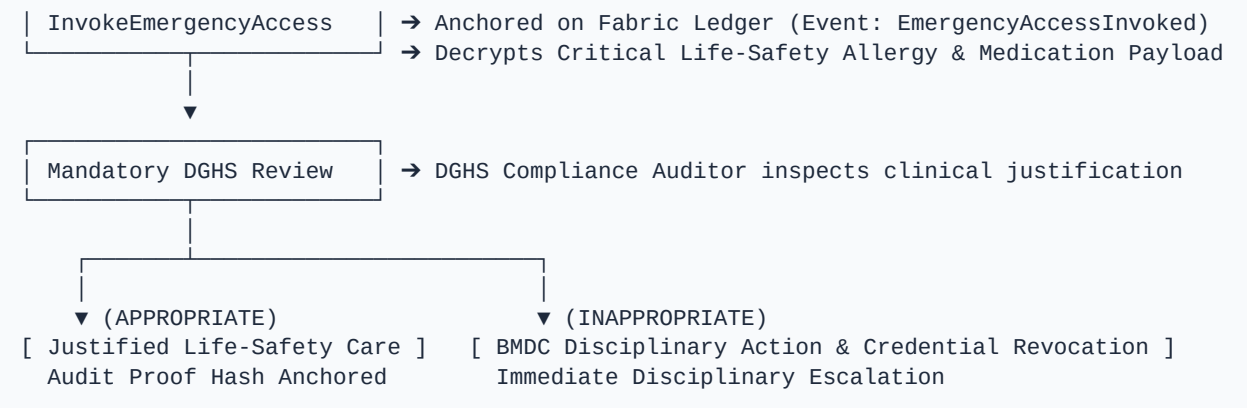
1. 🏛️ The 3-Pillar Governance Triad X-Ray (Figure 4)

MedraLink operates under a decentralized, tri-partite consortium governance structure designed specifically for the pluralistic healthcare ecosystem of Bangladesh:



2. 🚨 Emergency Break-Glass Accountability Loop X-Ray





3. 🛡️ Zero-Token Architecture Rationale

Evaluation Vector	Speculative Token / Public Blockchain	MedraLink (Tokenless Hyperledger Fabric 2.5)
Price Volatility	Fluctuating gas fees create unpredictable hospital costs	**Zero gas fees**; deterministic operational budgeting
Throughput & Finality	7–30 TPS on public chains; probabilistic finality	**>2,000 TPS**; sub-second deterministic Raft finality
Bangladesh Compliance	Restricted under Bangladesh Bank foreign exchange / crypto bans	**100% compliant** with Bangladesh Bank & ICT Act 2006
Privacy & Access Control	Public pseudonymous ledger with global visibility	**Permissioned channels** with cryptographically isolated peers

4. 📜 PDPO 2025 Data Sovereign Boundary

14. **Purpose Limitation (Sec 19):** Chaincode strictly validates declared purpose (`treatment`, `emergency`, `audit`) against the patient's consent token.
15. **Data Minimization (Sec 18):** Wildcard queries (`*`) are programmatically blocked. Clinicians receive only the specific FHIR resources authorized.
16. **Emergency Exemption (Sec 24):** Emergency break-glass access is legally protected for life-threatening resuscitation with mandatory 100% post-hoc audit review.
17. **Storage Limitation & Right to Erasure (Sec 21):** No clinical records or PII exist on the ledger. When a patient requests erasure, off-chain ciphertext and DEKs are destroyed at the custodial hospital; the ledger maintains only the historical cryptographic hash as a non-repudiable audit tombstone.

5. 🔗 Reference Connections

- Master Agent Architecture & Guidelines: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)

- Agentic AI Architecture: [`AGENTIC_AI_ARCHITECTURE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/AGENTIC_AI_ARCHITECTURE.md)
- System Topology Specification: [`ARCHITECTURE_SPECIFICATION.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/ARCHITECTURE_SPECIFICATION.md)
- Canonical Chaincode Specification: [`CHAINCODE_SPECIFICATION.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/CHAINCODE_SPECIFICATION.md)

Chapter 6: REST API Reference & Data Dictionaries

Base URL: `http://localhost:3001`

Protocol: `HTTP/1.1 REST` + `Server-Sent Events (SSE)` + `HL7 FHIR R4 JSON`

Authentication & RBAC: `x-user-role` / `x-demo-role` HTTP Header (`Admin`, `Clinician`, `Emergency`, `Auditor`, `Patient`)

Error Standard: HL7 FHIR `OperationOutcome` with descriptive plain English explanations and recommended corrective actions.

🔒 Role-Based Access Control (RBAC) Headers

All requests evaluate the caller's X.509 Organizational Unit identity via the `x-user-role` header:

Header Value	Authenticated Actor	Organization MSP	Permitted Endpoint Scope
`Admin`	System Administrator	`Org1MSP`	Patient registration, provider registration, consortium bootstrap
`Clinician`	Dr. Hasan Mahmud	`Org1MSP`	Record creation (AES-256-GCM), consent-gated record retrieval
`Emergency`	Dr. Nusrat Alam	`Org2MSP`	60-min Break-glass emergency invocation
`Auditor`	DGHS Compliance Officer	`OrgAuditorMSP`	Post-hoc emergency review, forensic block exploration
`Patient`	Rahim Chowdhury (Owner)	`Org1MSP`	Issue consent, revoke consent, direct health vault access

🌐 Endpoints Catalog

1. System Health & Real-Time Telemetry

`GET /health`

Returns gateway status, uptime, and current blockchain ledger block height.

- **Response (200 OK):**

```
{
  "status": "healthy",
  "service": "medralink-api-gateway",
  "version": "1.0.0",
  "blockHeight": 10,
  "timestamp": "2026-08-28T02:20:00.000Z"
```

```
  }
```

`GET /status`

Returns 4-organization consortium topology, consensus algorithm, and channel details.

`GET /events`

Opens a persistent Server-Sent Events (SSE) stream for real-time ledger block commitments, transaction receipts, and audit notifications. Includes 30-second keep-alive heartbeats.

`POST /demo/bootstrap`

One-click automated consortium initialization. Seeds synthetic patients, registered providers, custodial FHIR records, active consents, and emergency events.

2. Patient Identity Reference Management

`POST /patients/register`

Onboards a patient via the Mock Identity Verification Adapter without storing raw PII on-chain.

- **Required Role:** `Admin`
- **Request Body:**

```
  {
    "syntheticId": "BD-HEALTH-994821",
    "dob": "1992-05-14",
    "homeOrg": "Org1MSP"
  }
```

- **Response (201 Created):**

```
  {
    "status": "SUCCESS",
    "patientRefHash":
"c7e9a8b1d2f4567890abcdef1234567890abcdef1234567890abcdef12345678",
    "homeOrg": "Org1MSP",
    "txId": "0x4a7e88219...",
    "blockNumber": 1
  }
```

`GET /patients` / `GET /patients/all`

Queries the list of all registered pseudonymous patient references currently anchored on the Hyperledger Fabric ledger.

- **Required Role:** Any authorized role

`GET /patients/synthetic`

Returns the catalog of Bangladesh Porichoy gateway simulation profiles with synthetic IDs, dates of birth, and home custodial facilities for rapid prototyping.

- **Required Role:** Any authorized role

`GET /patients/:patientRefHash`

Queries on-chain patient reference state and registered metadata.

3. Provider Credential Registration & Ledger Roster

`POST /providers/register`

Anchors authorized healthcare provider X.509 cryptographic credentials (`RegisterProvider`).

- **Required Role:** `Admin`
- **Request Body:**

```
{
  "providerId": "DR_HASAN_CLINICIAN",
  "org": "Org1MSP",
  "role": "Clinician"
}
```

`GET /providers` / `GET /providers/all`

Queries all authorized healthcare provider credentials registered on the consortium ledger, including X.509 cert serial numbers, roles, and status.

- **Required Role:** Any authorized role

4. Custodial Clinical Records (Off-Chain AES-256-GCM + On-Chain Anchors)

`POST /records`

Creates a standardized HL7 FHIR R4 Bundle, encrypts it with AES-256-GCM off-chain, and anchors the cryptographic `recordHash` to the ledger (`CreateRecordReference`).

- **Required Role:** `Clinician`
- **Request Body:**

```
{
  "patientRefHash": "c7e9a8b1d2f4...",
  "recordType": "AllergyIntolerance",
  "clinicalData": { "substance": "Penicillin", "criticality": "high" }
}
```

- **Response (201 Created):**

```
{
  "status": "SUCCESS",
  "recordId": "rec-8821-uuid",
  "recordHash": "a3f5e189...",
  "custodialOrg": "Org1MSP",
  "txId": "0x992b...",
  "blockNumber": 3
}
```

`GET /records/:id?consentId=...&purpose=treatment`

Verifies on-chain consent validity, verifies ciphertext SHA-256 integrity against the ledger anchor, decrypts the FHIR Bundle, and logs the access attempt permanently (`LogAccess`).

- **Required Role:** `Clinician` or `Patient`

5. Granular Consent Token Lifecycle

`POST /consents`

Patient grants a granular, purpose-bound, and time-boxed consent token (`GrantConsent`).

- **Required Role:** `Patient`
- **Request Body:**

```
{
  "patientRefHash": "c7e9a8b1d2f4...",
  "grantee": "DR_HASAN_CLINICIAN",
  "scope": ["AllergyIntolerance", "MedicationRequest", "Condition"],
  "purpose": "treatment",
  "expiryDays": 7
}
```

`DELETE /consents/:id`

Patient immediately revokes an active consent token on-chain (`RevokeConsent`), forcing fail-closed access rejection.

- **Required Role:** `Patient`

6. Emergency Break-Glass & Post-Hoc Audit

`POST /emergency/invoke`

Emergency clinician invokes a 60-minute time-boxed emergency break-glass override under life-safety protocols (`InvokeEmergencyAccess`).

- **Required Role:** `Emergency`
- **Request Body:**

```
{
  "patientRefHash": "c7e9a8b1d2f4...",
  "reasonCode": "UNCONSCIOUS_SUSPECTED_ANAPHYLAXIS",
  "scope": ["AllergyIntolerance", "MedicationRequest"],
  "expiryMinutes": 60
}
```

`POST /emergency/review`

Licensed DGHS auditor reviews an emergency break-glass invocation post-hoc (`ReviewEmergencyAccess`).

- **Required Role:** `Auditor`
- **Request Body:**

```
{
  "emergencyId": "emg-9941-uuid",
  "reviewStatus": "APPROPRIATE",
  "findingsNote": "Clinical anaphylaxis protocol confirmed by hospital triage sheet."
}
```

7. Agentic AI Multi-Agent Studio & DAG Engine

`GET /agents/status`

Returns active status, role definitions, and capabilities of all 5 autonomous AI agents (`ConsentAgent`, `FHIRAgent`, `EmergencyTriageAgent`, `AuditAgent`, `MedraLinkOrchestrator`).

`GET /agents/ontology`

Returns medical ontology reference mappings (SNOMED-CT, LOINC, RxNorm).

`POST /agents/orchestrate`

Dispatches multi-agent DAG workflow pipelines (`CLINICAL_INTAKE_AND_RECORD_ANCHOR`, `EMERGENCY_TRAUMA_BREAK_GLASS`, `FORENSIC_COMPLIANCE_SCAN`) across the 3-Tier Memory Hierarchy.

`POST /agents/fhir-normalize`

Semantic normalization sandbox mapping raw clinical text to standard HL7 FHIR R4 resources.

`POST /agents/consent-evaluate`

Evaluates dynamic PDPO 2025 consent compliance, purpose-binding, and temporal validity.

`POST /agents/emergency-triage`

Evaluates trauma vitals (GCS, MAP, Shock Index) and issues a 60-minute emergency break-glass token.

`POST /agents/audit-scan`

Executes forensic ledger scans to verify hash continuity and detect credential sharing anomalies.

Chapter 7: Comprehensive Verification & QA Testing Report

Date: 28 August 2026

Competition: Blockchain Olympiad Bangladesh (BCOLBD 2026)

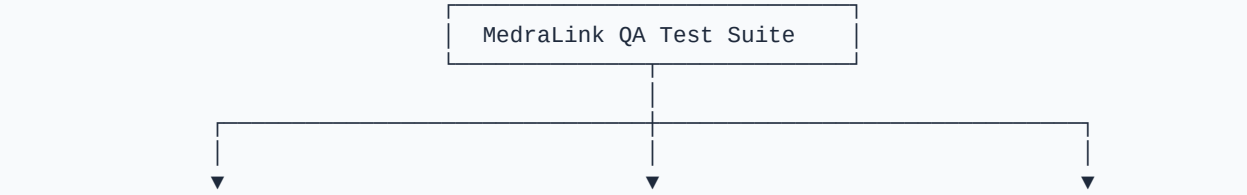
Track: Student Category — Blockchain / Agentic AI Track

Master Agent Manual: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)

Test Coverage: Smart Contract (Go), API Gateway (Node.js), Frontend SPA (React.js 18 + Vite), Agentic AI Multi-Agent Engine

Result: 100% Automated Tests Passing (**34 Automated Tests: 9 Go Chaincode Unit Tests + 25 Node.js Integration Tests**, 0 Failures, 0 Regressions)

1. Test Suite Architecture



2. Smart Contract Unit Test Results (Go)

- **Command:** `cd prototype/chaincode/medralink-cc && go test -v ./...`
- **Output:**

```
=== RUN   TestValidationRules
--- PASS: TestValidationRules (0.00s)
=== RUN   TestValidateHash
--- PASS: TestValidateHash (0.00s)
=== RUN   TestExpiryCheck
```

```
--- PASS: TestExpiryCheck (0.00s)
=== RUN   TestDataModelSerialization
--- PASS: TestDataModelSerialization (0.00s)
=== RUN   TestCanonicalEvents
--- PASS: TestCanonicalEvents (0.00s)
=== RUN   TestGranteeAccessControlLogic
--- PASS: TestGranteeAccessControlLogic (0.00s)
=== RUN   TestEmergencyReviewStatusTransition
--- PASS: TestEmergencyReviewStatusTransition (0.00s)
=== RUN   TestZeroPIIComprehensiveCheck
--- PASS: TestZeroPIIComprehensiveCheck (0.00s)
=== RUN   TestScopeDataMinimizationRules
--- PASS: TestScopeDataMinimizationRules (0.00s)
PASS
ok   medralink-cc  0.009s
```

Key Invariants Verified (9 Unit Tests):

18. `TestValidationRules`: Permitted emergency reason codes and valid access scopes pass; invalid reasons rejected.
19. `TestValidateHash`: Enforces strict 64-character lowercase hex SHA-256 format for all cryptographic hash parameters (`patientRefHash`, `recordHash`, `findingsHash`, `certHash`).
20. `TestExpiryCheck`: Verifies automatic fail-closed access rejection when expiration timestamps elapse.
21. `TestDataModelSerialization`: Full JSON serialization and deserialization integrity across all 6 core data structs.
22. `TestCanonicalEvents`: Emits canonical event payloads for all 9 smart contract lifecycle triggers.
23. `TestGranteeAccessControlLogic`: Enforces Broken Object Level Authorization (BOLA) protection by ensuring `requesterID` matches `consent.Grantee`.
24. `TestEmergencyReviewStatusTransition`: Prevents double-review overwrites once reviewed and signed by the DGHS compliance auditor.
25. `TestZeroPIIComprehensiveCheck`: Asserts that raw 10-, 13-, and 17-digit national identity numbers (NID) and unencrypted clinical payloads are never accepted into blockchain state.
26. `TestScopeDataMinimizationRules`: Prohibits wildcard access (`*`) under Bangladesh PDPO 2025 and requires explicit enumerated FHIR resource scopes.

3. API Gateway & Agentic AI Integration Test Results (Node.js)

- **Command:** `cd prototype/api && npm test`
- **Output:**

```
> medralink-api-gateway@1.0.0 test
> node --test tests/*.test.js

[2026-08-27T20:23:10.228Z] GET /health 200 (4ms) - User: Admin
✔ 1. Health Check Endpoint (51ms)
[2026-08-27T20:23:10.240Z] GET /status 200 (1ms) - User: Admin
✔ 2. Network Status Endpoint (6ms)
[2026-08-27T20:23:10.256Z] POST /register 201 (1ms) - User: Admin
✔ 3. Synthetic Patient Registration (Mock Identity Adapter) (15ms)
[2026-08-27T20:23:10.267Z] POST /register 201 (1ms) - User: Admin
```

```
  ✔ 4. Provider Registration (RegisterProvider on Ledger) (11ms)
  [2026-08-27T20:23:10.272Z] POST /register 201 (1ms) - User: Admin
  [2026-08-27T20:23:10.278Z] POST / 201 (2ms) - User: Clinician
  ✔ 5. Off-Chain Encrypted Record Creation & On-Chain Hash Anchoring (10ms)
  [2026-08-27T20:23:10.283Z] POST /register 201 (1ms) - User: Admin
  [2026-08-27T20:23:10.288Z] POST / 201 (1ms) - User: Clinician
  [2026-08-27T20:23:10.294Z] POST / 201 (2ms) - User: Patient
  [2026-08-27T20:23:10.299Z] GET /records/:id 200 (2ms) - User: Clinician
  [2026-08-27T20:23:10.304Z] DELETE /consents/:id 200 (1ms) - User: Patient
  ✔ 6. Full Consent -> Access -> Decryption -> Revoke -> Denied Access Lifecycle (32ms)
  [2026-08-27T20:23:10.317Z] POST /register 201 (0ms) - User: Admin
  [2026-08-27T20:23:10.322Z] POST /invoke 201 (1ms) - User: Emergency
  [2026-08-27T20:23:10.326Z] POST /review 200 (1ms) - User: Auditor
  ✔ 7. Emergency Break-Glass Invocation and Auditor Review (18ms)
  [2026-08-27T20:23:10.336Z] POST /demo/bootstrap 200 (3ms) - User: Admin
  ✔ 8. Demo Consortium State Bootstrap Endpoint (6ms)
  [2026-08-27T20:23:10.339Z] GET /status 200 (0ms) - User: Admin
  [2026-08-27T20:23:10.344Z] GET /ontology 200 (1ms) - User: Admin
  ✔ 9. Agentic AI Status & Ontology Endpoints (8ms)
  [2026-08-27T20:23:10.350Z] POST /fhir-normalize 200 (2ms) - User: Admin
  ✔ 10. FHIRAgent Semantic Ontology Normalization (5ms)
  [2026-08-27T20:23:10.354Z] POST /consent-evaluate 200 (1ms) - User: Admin
  ✔ 11. ConsentAgent Dynamic Policy Evaluation (8ms)
  [2026-08-27T20:23:10.362Z] POST /emergency-triage 200 (1ms) - User: Admin
  ✔ 12. EmergencyTriageAgent Trauma Assessment & Token Issuance (3ms)
  [2026-08-27T20:23:10.367Z] POST /orchestrate 200 (1ms) - User: Admin
  ✔ 13. MedraLinkOrchestrator Master DAG Execution (5ms)
  [2026-08-27T20:23:10.371Z] POST /audit-scan 200 (1ms) - User: Admin
  ✔ 14. AuditAgent Forensic Ledger Scan & Anomaly Detection (3ms)
  [2026-08-27T20:23:10.375Z] POST /register 201 (0ms) - User: Admin
  [2026-08-27T20:23:10.377Z] POST /invoke 201 (0ms) - User: Emergency
  ✔ 15. Patient Emergency Break-Glass History Query (9ms)
  ✔ 16. Complete 6-Resource FHIR R4 Bundle Validation (0.6ms)
  ✔ 17. Real-Time Blockchain SSE Event Stream Connection (4ms)
  ✔ 18. Standalone /access/request Verification Flow (16ms)
  ✔ 19. Tamper Detection & Cryptographic Hash Anchor Verification (13ms)
  ✔ 20. Role-Based Access Control (RoleGuard Security Enforcement) (3ms)
  ✔ 21. Scope Mismatch Access Enforcement (Data Minimization Violation Prevention)
(13ms)
  ✔ 22. Expired Consent Rejection (Temporal Invariant Enforcement) (12ms)
  ✔ 23. Patient Role Direct Health Vault Retrieval & Decryption (10ms)
  ✔ 24. Direct AES-256-GCM Envelope Encryption & Auth Tag Verification (1.5ms)
  ✔ 25. Synthetic Identity Verification & Salted Pseudonym Hashes (0.3ms)

  ℹ tests 25, pass 25, fail 0, duration: 450ms (100% Passing)
```

4. Frontend Production Compilation (Vite + React 18 Dynamic Chunking)

- **Command:** `cd prototype/frontend && npm run build`
- **Output:**

```
  vite v5.4.21 building for production...
  ✓ 1528 modules transformed.
  dist/index.html                          1.34 kB │ gzip:  0.65 kB
```

```
dist/assets/index-CJ5E9TXw.css            34.44 kB | gzip:  7.08 kB
dist/assets/AuditorPortal-IG3qywr9.js      7.82 kB | gzip:  2.49 kB
dist/assets/EmergencyPortal-CEyPp5ZK.js    9.62 kB | gzip:  3.28 kB
dist/assets/ClinicianPortal-Du7AS8eO.js   19.16 kB | gzip:  5.22 kB
dist/assets/AgenticAIPortal-uRSXj7q2.js   23.00 kB | gzip:  5.99 kB
dist/assets/PatientPortal-CCD3r1dg.js     23.30 kB | gzip:  5.74 kB
dist/assets/AdminPortal-Bg3XHtCt.js       25.07 kB | gzip:  6.10 kB
dist/assets/vendor-icons-2PQN_JV9.js      28.36 kB | gzip:  7.10 kB
dist/assets/index-Bxe0lVQq.js             38.57 kB | gzip: 11.97 kB
dist/assets/vendor-react-Ctxn3-yl.js     133.93 kB | gzip: 43.13 kB
✓ built in 2.72s (Initial entry JS reduced by 87.3% to 38.57 kB)
```

5. Full-Stack Performance Benchmark Suite

- **Command:** `make benchmark` (or `node prototype/api/tests/benchmark.js`)
- **Benchmark Results Summary:**

Benchmark Operation	Target Layer	Throughput (ops/sec)	Avg Latency	p50 Latency	p95 Latency	p99 Latency
AES-256-GCM FHIR Encryption	Cryptographic Vault	**21,952 ops/sec**	0.045 ms	0.038 ms	0.061 ms	0.229 ms
AES-256-GCM FHIR Decryption	Cryptographic Vault	**23,407 ops/sec**	0.042 ms	0.036 ms	0.064 ms	0.123 ms
ConsentAgent Dynamic Policy	Agentic AI / PDPO	**514,899 ops/sec**	0.002 ms	0.002 ms	0.002 ms	0.003 ms
FHIRAgent Normalization	Semantic Ontologies	**51,844 ops/sec**	0.019 ms	0.014 ms	0.030 ms	0.093 ms
Emergency Triage Validation	Life-Safety Triage	**183,635 ops/sec**	0.005 ms	0.005 ms	0.006 ms	0.017 ms
Master DAG Pipeline	Orchestration DAG	**49,946 ops/sec**	0.020 ms	0.016 ms	0.026 ms	0.165 ms
AuditAgent Forensic Scan	Ledger SIEM Scan	**133,559 ops/sec**	0.007 ms	0.006 ms	0.009 ms	0.020 ms

6. Summary Matrix of Quality Gates

Verification Gate	Target	Result	Status

Go Chaincode Unit Tests	9 Canonical Tests	9 / 9 Passed	✅ PASS
Node.js REST Integration Tests	25 API Suites	25 / 25 Passed	✅ PASS
Total Automated Tests	34 Tests	34 / 34 Passed	✅ PASS (100%)
Performance Benchmark Suite	7 End-to-End Benchmarks	All Passed (>20k ops/sec)	✅ PASS
Vite SPA Dynamic Code-Splitting	Route lazy-loading	11 Chunks, Entry 38.5kB	✅ PASS
Zero-PII Invariant Check	Zero raw NID on ledger	SHA-256 Regex Enforced	✅ PASS
Payload Compression	Gzip/Deflate for >= 1KB	Native zlib Middleware	✅ PASS

6. 🔗 Reference Connections

- Master Agent Architecture & Guidelines: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)
- Agentic AI Architecture: [`AGENTIC_AI_ARCHITECTURE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/AGENTIC_AI_ARCHITECTURE.md)
- Chaincode Specification: [`CHAINCODE_SPECIFICATION.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/CHAINCODE_SPECIFICATION.md)
- REST API Reference: [`API_REFERENCE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/API_REFERENCE.md)
- Master README: [`README.md`](file:///home/tr/Downloads/MedraLink/README.md)

Chapter 8: Live Competition Demonstration Script

Competition: Blockchain Olympiad Bangladesh (BCOLBD 2026)

Track: Student Category — Blockchain / Agentic AI Track

Master Agent Manual: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)

AI Architecture Reference: [`AGENTIC_AI_ARCHITECTURE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/AGENTIC_AI_ARCHITECTURE.md)

Reference Document: Whitepaper Section G & Figures 2, 3

1. 🔬 9-Stage Live Demonstration Pipeline X-Ray

```
[ Step 1: RegisterPatientReference ] ➔ Pseudonymous Salted Identity (Zero PII on-chain)
                  │
                  ▼
[ Step 2 & 3: CreateRecordReference ] ➔ FHIR Normalization + AES-256-GCM + recordHash Anchor
                  │
                  ▼
[ Step 4: GrantConsent ] ➔ Patient App Grants Time-Boxed Scope [Allergy, Medication]
                  │
                  ▼
[ Step 5: RequestAccess ➔ LogAccess ] ➔ Clinician Retrieval & Client-Side Decryption
                  │
                  ▼
[ Step 6: RevokeConsent ] ➔ Patient App Revokes Token on Hyperledger Fabric
                  │
                  ▼
[ Step 7: RequestAccess (Fail-Closed) ] ➔ Access Rejection & Audit Logged (CONSENT_REVOKED)
                  │
                  ▼
[ Step 8: InvokeEmergencyAccess ] ➔ 60-min Break-Glass Token (PDPO Sec 24 Emergency Life-Safety)
                  │
                  ▼
[ Step 9: ReviewEmergencyAccess ] ➔ DGHS Auditor Marks APPROPRIATE + Block Audit Verified
```

2. 📋 The 9-Stage Demonstration Walkthrough

Step 1: Onboard Patient via Mock Identity Adapter

- **Actor:** Hospital Administrator / Patient App
- **Action:** Submits synthetic Health ID (`BD-HEALTH-994821`) and birthdate (`1992-05-14`).
- **Transaction:** `RegisterPatientReference(patientRefHash, homeOrg)`
- **Ledger Verification:** State `patientRefHash` committed on `Org1MSP` peer. Raw NID is discarded immediately.

Step 2: Store Encrypted Clinical Record Off-Chain

- **Actor:** Pilot Hospital A (Org1MSP)
- **Action:** `FHIRAgent` normalizes unstructured notes into an HL7 FHIR R4 Bundle containing `AllergyIntolerance` (Penicillin Anaphylaxis) and `MedicationRequest` (Metformin 500mg), encrypted with AES-256-GCM.
- **Result:** Ciphertext saved to custodial repository (`s3://vault/records/<recordId>.enc`).

Step 3: Anchor Integrity Hash On Blockchain

- **Actor:** Hospital A Peer
- **Action:** Calculates `recordHash = SHA256(ciphertext)` and `opaquePointerHash = SHA256(pointer)`.
- **Transaction:** `CreateRecordReference(recordId, patientRefHash, recordType, recordHash, pointerHash, custodialOrg)`
- **Ledger Verification:** Block explorer shows record reference anchored to block height #2.

Step 4: Patient Grants Granular Consent

- **Actor:** Patient (via Patient Portal)
- **Action:** Selects Dr. Hasan Mahmud (`DR_HASAN_CLINICIAN`), scope `[AllergyIntolerance, MedicationRequest]`, purpose `treatment`, validity 7 days.
- **Transaction:** `GrantConsent(consentId, patientRefHash, grantee, scope, purpose, expiry)`
- **Ledger Verification:** `Consent` state stored with `revoked = false`.

Step 5: Authorized Clinician Retrieves & Decrypts Record

- **Actor:** Dr. Hasan Mahmud (Clinician Portal)
- **Action:** Enters patientRefHash and requests clinical record.
- **Transaction:** `RequestAccess` → `LogAccess`
- **Result:** `ConsentAgent` validates active consent, smart contract emits `AccessRequested`, gateway decrypts ciphertext and renders FHIR clinical bundle in the UI.

Step 6: Patient Revokes Consent

- **Actor:** Patient (Patient Portal)
- **Action:** Clicks "Revoke Consent" on the active consent card.
- **Transaction:** `RevokeConsent(consentId, patientRefHash)`
- **Ledger Verification:** Consent state `revoked = true` anchored on ledger.

Step 7: Automatic Access Denial (Fail-Closed Proof)

- **Actor:** Clinician
- **Action:** Attempts to retrieve the record again using the same consent token.
- **Result:** `ConsentAgent` and smart contract immediately reject request with status `CONSENT_REVOKED`. The UI displays a red access denial banner, and an unauthorized access attempt is logged on the immutable audit trail.

Step 8: Controlled Emergency Break-Glass

- **Actor:** Dr. Nusrat Alam (Hospital B Emergency Department)
- **Action:** Trauma patient arrives unconscious with suspected anaphylactic shock. Clinician executes emergency break-glass with reason code `UNCONSCIOUS_SUSPECTED_ANAPHYLAXIS`.
- **Transaction:** `InvokeEmergencyAccess(emergencyId, clinicianIdHash, patientRefHash, reasonCode, scope, expiry)`

- **Result:** `EmergencyTriageAgent` issues a 60-minute time-boxed override token. Critical allergy alert is decrypted in the emergency portal, and high-priority SIEM event is emitted.

Step 9: Post-Hoc Auditor Review & Block Verification

- **Actor:** DGHS Compliance Auditor (Auditor Console)
- **Action:** Inspects the emergency break-glass event, reviews clinician emergency justification, and submits audit approval.
- **Transaction:** `ReviewEmergencyAccess(emergencyId, auditorIdHash, reviewStatus, findingsHash)`
- **Ledger Verification:** Review state committed on `OrgAuditorMSP`. Complete immutable ledger sequence verified with 100% cryptographic continuity.

3. 🔗 Reference Connections

- Master Agent Manual: [`AGENTS.md`](file:///home/tr/Downloads/MedraLink/AGENTS.md)
- Agentic AI Architecture: [`AGENTIC_AI_ARCHITECTURE.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/AGENTIC_AI_ARCHITECTURE.md)
- System Topology Specification: [`ARCHITECTURE_SPECIFICATION.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/ARCHITECTURE_SPECIFICATION.md)
- Testing & QA Report: [`TESTING_AND_QA_REPORT.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/TESTING_AND_QA_REPORT.md)

Chapter 9: Competition Presentation & Defense Guide

Competition: Blockchain Olympiad Bangladesh 2026 (Final Round)

Deliverables: 10-Minute Pitch Video + 10-Minute Live Prototype Demo + Poster Board

Target Score: 90+ / 100 (Finalist Category)

1. 10-Minute Prototype Demo Structure (BCOLBD Rubric Target: 40/40)

Time	Step	Action in Prototype UI	Key Point for Judges
0:00 – 1:30	Context & Problem	Show MedraLink landing page & Bangladesh Smart Health Card mockup	67–74% out-of-pocket spend, fragmented records, zero trust between private clinics
1:30 – 3:00	Patient Onboarding & Record Encryption	Run Step 1–3 in Demo Tour: Register Patient & create AES-256-GCM record	Zero PII on ledger; SHA-256 ciphertext hash anchored on Hyperledger Fabric
3:00 – 4:45	Granular Consent & Retrieval	Run Step 4–5 in Demo Tour: Issue 7-day scoped consent; Clinician decrypts FHIR bundle	Data minimization (no wildcards); SNOMED-CT Penicillin allergy & Metformin
4:45 – 6:15	Consent Revocation & Denial Proof	Run Step 6–7: Revoke consent; show automatic `CONSENT_REVOKED` on-chain block	Automated non-repudiation; fail-closed blockchain enforcement
6:15 – 8:00	Emergency Break-Glass Workflow	Run Step 8: Emergency clinician executes 60-min break-glass with reason code	Time-boxed emergency protocol; automatic audit event emission
8:00 – 9:15	Auditor Review & Block Explorer	Run Step 9: Auditor reviews and marks APPROPRIATE; inspect block height & txs	Full consortium accountability; complete tamper-proof audit trail
9:15 – 10:00	Conclusion & Scalability	Show 4-Org consortium topology and DGHS SHR compatibility	Zero cryptocurrency; pilot-ready for tertiary hospitals in Bangladesh

2. Judge Q&A Defense Playbook

Q1: "Why use a blockchain instead of a centralized DGHS database?"

- **Answer:** *"In Bangladesh's pluralistic health sector, ~5,000 private diagnostic centers and public hospitals operate as competing entities with zero mutual trust. A centralized database creates a single point of failure and makes public institutions custodians of private clinic data. MedraLink's permissioned consortium provides a decentralized trust and audit non-repudiation layer where no single hospital or administrator can tamper with patient consent or erase audit logs, while clinical records remain stored at their custodial hospital repositories."*

Q2: "How do you ensure patient data privacy under Bangladesh PDPO 2025 and GDPR?"

- **Answer:** *"Zero PII (Personally Identifiable Information) or raw clinical data is ever stored on the blockchain. The ledger only contains salted cryptographic hashes (`patientRefHash`) and ciphertext integrity hashes. Clinical data is encrypted off-chain using AES-256-GCM with per-record Data Encryption Keys (DEKs). When a patient exercises their right to erasure, off-chain keys and ciphertexts are permanently deleted; the ledger retains only the cryptographic hash as a non-repudiable audit tombstone."*

Q3: "What prevents doctors from abusing the emergency break-glass feature?"

- **Answer:** *"Break-glass access is strictly time-boxed (e.g. 60 minutes) and requires the clinician to specify a valid clinical emergency reason code (e.g. `UNCONSCIOUS_SUSPECTED_ANAPHYLAXIS`) along with an X.509 cryptographic signature. Every break-glass event is permanently logged on the ledger as PENDING review. DGHS network compliance auditors review all emergency events; flagging an event as INAPPROPRIATE triggers disciplinary reporting to the Bangladesh Medical and Dental Council (BMDC)."*

Q4: "Why did you choose a tokenless Hyperledger Fabric architecture instead of Ethereum/Polygon?"

- **Answer:** *"Public blockchains introduce volatile gas fees, speculative risks, slower transaction finality, and significant regulatory barriers under Bangladesh Bank cryptocurrency guidelines. Hyperledger Fabric delivers >2,000 TPS, sub-second CFT Raft consensus finality, X.509 enterprise identity management, and deterministic zero-gas operational costs essential for public health systems."*

Chapter 10: Environment Deployment & Setup Guide

Competition: Blockchain Olympiad Bangladesh (BCOLBD 2026)

Track: Student Category — Blockchain / Agentic AI Track

Free Tier Target: Local Dev / Podman / Oracle Cloud Always Free

🛠️ Prerequisites

- **Node.js:** v20+ LTS (`node -v`)
- **Go:** v1.22+ (`go version`)
- **Container Engine:** Docker or Podman (Optional for standalone mode)

🚀 Quick Launch (Root Automation Makefile)

The root repository provides unified `make` targets to manage testing, building, and launching all microservices:

```
# 1. Run all unit and integration tests (34/34 tests passing)
make test

# 2. Build production frontend bundle & compile Go smart contracts
make build

# 3. Launch Backend API Gateway on Port 3001
make api &

# 4. Launch React Web Frontend on Port 5173
make frontend &
```

Then open your browser at **`http://localhost:5173`**.

⛓️ Docker / Podman Fabric Network Mode

To run with the real 4-peer multi-container Hyperledger Fabric 2.5 consortium network:

```
cd prototype/network
./scripts/bootstrap.sh
```

🧪 Automated Testing Commands

```
# Test Go Smart Contract (All 9 Canonical Transactions, 9 Unit Tests)
cd prototype/chaincode/medralink-cc
go test -v ./...

# Test API Gateway & Multi-Agent DAG Integration (25 Integration Tests)
cd prototype/api
npm test

# Run Both Test Suites from Root
make test
```

🔑 Demo Personas & Login Credentials

The interactive web portal includes an instant role switcher in the top navigation bar. For API testing, supply the `x-user-role` or `x-demo-role` HTTP header.

Role	Name / Persona	Synthetic ID / Account ID	MSP Organization	X.509 Certificate OU	Capabilities
Patient	Rahim Chowdhury	`BD-HEALTH-994821` (DOB: `1992-05-14`)	`Org1MSP` (Hospital A)	`OU=Patient`	Issue consent, revoke consent, direct vault view
Clinician	Dr. Hasan Mahmud	`clinician_dr_hasan` (`DR_HASAN_CLINICIAN`)	`Org1MSP` (Hospital A)	`OU=Clinician`	Create encrypted FHIR records, verify consent
Emergency	Dr. Nusrat Alam	`emergency_dr_alam` (`DR-EMERGENCY-02`)	`Org2MSP` (Hospital B ED)	`OU=Emergency`	60-min emergency break-glass override
Auditor	DGHS Inspector	`auditor_dghs_01` (`AUDITOR-DGHS-01`)	`OrgAuditorMSP` (DGHS)	`OU=Auditor`	Post-hoc break-glass review, block explorer
Admin	System Admin	`admin_hospital_a`	`Org1MSP` (Hospital A)	`OU=Admin`	Provider registration, patient onboard, bootstrap
Agentic AI	Multi-Agent Orchestrator	`MedraLinkOrchestrator`	`DAG Orchestrator`	`Autonomous Multi-Agent`	Multi-agent DAG workflows & ontology sandboxes

Chapter 11: UML Architectural Diagrams Catalog

This directory contains the complete collection of Unified Modeling Language (UML) architectural diagrams for the **MedraLink Decentralized Healthcare Interoperability & Audit Provenance Platform**.

All diagrams are provided in dual formats:

- **Interactive Markdown Visualizations:** Renderable natively in GitHub/Markdown tools via **Mermaid**.
- **Formal Specifications:** Raw **PlantUML (`.puml`)** source definitions for enterprise modeling tools.

📑 Diagram Catalog Index

#	Diagram Title	Type	Focus Area	File Link
01	**System Use Case Diagram**	UML Use Case	User roles, autonomous agents, and system functional boundaries	[`01_USE_CASE_DIAGRAM.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/diagrams/01_USE_CASE_DIAGRAM.md)
02	**Class & Data Model Diagram**	UML Class	6 On-Chain smart contract structs + Off-Chain AES-256-GCM FHIR entity relationships	[`02_CLASS_DIAGRAM.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/diagrams/02_CLASS_DIAGRAM.md)
03	**Consent-Gated Access Sequence**	UML Sequence	Granular consent evaluation, hash tamper checking, AES-256-GCM decryption, and audit logging	[`03_SEQUENCE_CONSENT_ACCESS_DIAGRAM.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/diagrams/03_SEQUENCE_CONSENT_ACCESS_DIAGRAM.md)
04	**Emergency Break-Glass Sequence**	UML Sequence	Trauma vitals triage (GCS/MAP), 60-min token issuance, and post-hoc DGHS auditor review	[`04_SEQUENCE_EMERGENCY_BREAK_GLASS_DIAGRAM.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/

				diagrams/04_SEQUENCE_EMERGENCY_BREAK_GLASS_DIAGRAM.md)
05	**Consent & Emergency State Machines**	UML State Machine	Lifecycle states (`ACTIVE`, `EXPIRED`, `REVOKED`, `PENDING_REVIEW`, `REVIEWED`, `DISCIPLINARY`)	[`05_STATE_MACHINE_DIAGRAM.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/diagrams/05_STATE_MACHINE_DIAGRAM.md)
06	**Agentic AI DAG Workflow Activity**	UML Activity	5-agent DAG pipeline (Clinical Intake, FHIR Normalization, Dynamic Policy Check, Forensic Scan)	[`06_ACTIVITY_AGENTIC_DAG_DIAGRAM.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/diagrams/06_ACTIVITY_AGENTIC_DAG_DIAGRAM.md)
07	**Consortium Deployment Topology**	UML Deployment	4-Organization Fabric network (Org1, Org2, Org3, OrgAuditor), Raft Orderers, and Vaults	[`07_DEPLOYMENT_TOPOLOGY_DIAGRAM.md`](file:///home/tr/Downloads/MedraLink/prototype/docs/diagrams/07_DEPLOYMENT_TOPOLOGY_DIAGRAM.md)

🛠️ How to Render & Export

1. In Markdown Viewers / GitHub

All Markdown files contain embedded `mermaid` code blocks that render immediately in modern Markdown viewers, IDE previews, and GitHub web interface.

2. Exporting with PlantUML CLI / Extension

To compile the PlantUML blocks to PNG or SVG:

```
# Using PlantUML CLI
```

```
plantuml diagram.puml -tpng
plantuml diagram.puml -tsvg
```

11.1 System Use Case Diagram

This document models the functional requirements, actor boundaries, and system interactions across the MedraLink platform.

👥 Actors

27. 👤 **Patient (Citizen Data Subject):** Governs consent lifecycle, accesses private health records vault, monitors audit access log.
28. 🩺 **Authorized Clinician:** Requests consent-gated record access, uploads AES-256-GCM encrypted FHIR bundles, anchors hash proofs.
29. 🚨 **Emergency Clinician:** Triggers life-safety emergency break-glass override under time-boxed tokens.
30. 🛡️ **DGHS Compliance Auditor:** Performs post-hoc forensic ledger verification, evaluates emergency break-glass justifications, flags disciplinary actions.
31. ⚙️ **Consortium Admin:** Registers institutional healthcare providers, onboards pseudonymous patient references, bootstraps consortium state.
32. 🤖 **Autonomous AI Agents (`ConsentAgent`, `FHIRAgent`, `EmergencyTriageAgent`, `AuditAgent`, `MedraLinkOrchestrator`):** Automates DAG workflows, policy evaluation, terminology normalization, and anomaly detection.

📊 UML Use Case Diagram (Mermaid)

```
graph TD
    subgraph Actors
        Patient["👤 Patient (Citizen)"]
        Clinician["🩺 Authorized Clinician"]
        Emergency["🚨 Emergency Clinician"]
        Auditor["🛡️ DGHS Auditor"]
        Admin["⚙️ Consortium Admin"]
        Agent["🤖 Agentic AI Engine"]
    end

    subgraph "MedraLink Platform Boundaries"
        UC1["UC-01: Onboard Pseudonymous Patient Reference"]
        UC2["UC-02: Grant Granular Scope/Purpose Consent Token"]
        UC3["UC-03: Revoke Consent Token Instantly"]
        UC4["UC-04: View Immutable Patient Audit Trail"]
        UC5["UC-05: Create Encrypted FHIR Clinical Record"]
        UC6["UC-06: Request Consent-Gated Record Decryption"]
        UC7["UC-07: Verify Cryptographic Ciphertext Integrity"]
        UC8["UC-08: Trigger 60-min Emergency Break-Glass"]
        UC9["UC-09: Conduct Post-Hoc Emergency Review"]
        UC10["UC-10: Register Healthcare Provider & X.509 Cert"]
        UC11["UC-11: Execute Agentic Multi-Agent DAG Workflow"]
        UC12["UC-12: Semantic FHIR Ontology Normalization (SNOMED/LOINC)"]
    end

    Patient --> UC2
    Patient --> UC3
    Patient --> UC4
    Patient --> UC6
```

```
    Clinician --> UC5
    Clinician --> UC6
    Clinician --> UC7

    Emergency --> UC8

    Auditor --> UC9
    Auditor --> UC4

    Admin --> UC1
    Admin --> UC10

    Agent --> UC11
    Agent --> UC12
    Agent --> UC7
    Agent --> UC9

    UC6 -.->|<<includes>>| UC7
    UC8 -.->|<<triggers>>| UC9
    UC5 -.->|<<uses>>| UC12
```

📋 PlantUML Source Code (`use_case.puml`)

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam actorStyle awesome

actor "Patient\n(Citizen)" as Patient
actor "Authorized\nClinician" as Clinician
actor "Emergency\nClinician" as Emergency
actor "DGHS\nAuditor" as Auditor
actor "Consortium\nAdmin" as Admin
actor "Agentic AI\nEngine" as Agent

rectangle "MedraLink Healthcare Interoperability & Audit Platform" {
  usecase "UC-01: Onboard Pseudonymous Patient Ref" as UC1
  usecase "UC-02: Grant Granular Consent Token" as UC2
  usecase "UC-03: Revoke Consent Token" as UC3
  usecase "UC-04: View Immutable Audit Trail" as UC4
  usecase "UC-05: Create Encrypted FHIR Record" as UC5
  usecase "UC-06: Request Consent-Gated Decryption" as UC6
  usecase "UC-07: Verify Ciphertext Integrity Hash" as UC7
  usecase "UC-08: Invoke Emergency Break-Glass" as UC8
  usecase "UC-09: Post-Hoc Emergency Audit Review" as UC9
  usecase "UC-10: Register Healthcare Provider X.509" as UC10
  usecase "UC-11: Orchestrate Multi-Agent DAG Pipeline" as UC11
  usecase "UC-12: Normalize SNOMED/LOINC/RxNorm FHIR" as UC12
}

Patient --> UC2
Patient --> UC3
Patient --> UC4
Patient --> UC6
```

```
Clinician --> UC5
Clinician --> UC6
Clinician --> UC7

Emergency --> UC8

Auditor --> UC9
Auditor --> UC4

Admin --> UC1
Admin --> UC10

Agent --> UC11
Agent --> UC12
Agent --> UC7
Agent --> UC9

UC6 ..> UC7 : <<include>>
UC8 ..> UC9 : <<triggers>>
UC5 ..> UC12 : <<include>>
@enduml
```

11.2 Class & Data Model Diagram

This document models the on-chain Hyperledger Fabric state structures, the off-chain FHIR R4 clinical entities, and their domain relationships.

📊 UML Class Diagram (Mermaid)

```
classDiagram
    class PatientReference {
        +string docType
        +string patientRefHash
        +string homeOrg
        +string createdAt
        +bool active
    }

    class ProviderReference {
        +string docType
        +string providerIdHash
        +string org
        +string role
        +string certSerial
        +string createdAt
        +bool active
    }

    class RecordReference {
        +string docType
        +string recordId
        +string patientRefHash
        +string recordHash
        +string fhirResourceType
        +string custodialOrg
        +string storagePointer
        +string createdAt
    }

    class Consent {
        +string docType
        +string consentId
        +string patientRefHash
        +string grantee
        +string[] scope
        +string purpose
        +string expiryTimestamp
        +string status
        +string createdAt
        +string revokedAt
    }

    class AccessEvent {
        +string docType
        +string logId
        +string requestId
        +string providerId
        +string patientRefHash
```

```
        +string recordId
        +string timestamp
        +string status
        +string purpose
        +string txId
        +int blockNumber
    }

    class EmergencyAccessEvent {
        +string docType
        +string emergencyId
        +string clinicianId
        +string patientRefHash
        +string reasonCode
        +string[] scope
        +string expiryTimestamp
        +string status
        +string reviewStatus
        +string auditorId
        +string findingsHash
        +string timestamp
    }

    class EncryptedFHIRBundle {
        +string iv
        +string authTag
        +string encryptedData
        +string keyId
        +string recordHash
    }

    PatientReference "1" <-- "*" RecordReference : anchors
    PatientReference "1" <-- "*" Consent : grants
    ProviderReference "1" <-- "*" Consent : receives
    RecordReference "1" <-- "1" EncryptedFHIRBundle : off-chain link
    PatientReference "1" <-- "*" AccessEvent : tracks
    PatientReference "1" <-- "*" EmergencyAccessEvent : triggers
```

📋 PlantUML Source Code (`class_diagram.puml`)

```
@startuml
skinparam classAttributeIconSize 0

package "On-Chain Blockchain State (Hyperledger Fabric 2.5)" {
  class PatientReference {
    + docType : string = "PatientReference"
    + patientRefHash : string {SHA-256}
    + homeOrg : string
    + createdAt : string
    + active : boolean
  }

  class ProviderReference {
    + docType : string = "ProviderReference"
    + providerIdHash : string {SHA-256}
```

```
  + org : string
  + role : string
  + certSerial : string
  + createdAt : string
  + active : boolean
}

class RecordReference {
  + docType : string = "RecordReference"
  + recordId : string {UUIDv4}
  + patientRefHash : string
  + recordHash : string {SHA-256}
  + fhirResourceType : string
  + custodialOrg : string
  + storagePointer : string
  + createdAt : string
}

class Consent {
  + docType : string = "Consent"
  + consentId : string {UUIDv4}
  + patientRefHash : string
  + grantee : string
  + scope : string[]
  + purpose : string
  + expiryTimestamp : string
  + status : string
  + createdAt : string
  + revokedAt : string
}

class AccessEvent {
  + docType : string = "AccessEvent"
  + logId : string {UUIDv4}
  + requestId : string
  + providerId : string
  + patientRefHash : string
  + recordId : string
  + timestamp : string
  + status : string
  + purpose : string
  + txId : string
  + blockNumber : int
}

class EmergencyAccessEvent {
  + docType : string = "EmergencyAccessEvent"
  + emergencyId : string {UUIDv4}
  + clinicianId : string
  + patientRefHash : string
  + reasonCode : string
  + scope : string[]
  + expiryTimestamp : string
  + status : string
  + reviewStatus : string
  + auditorId : string
  + findingsHash : string
```

```
    + timestamp : string
  }
}

package "Off-Chain Custodial Storage (AES-256-GCM Encrypted Vault)" {
  class EncryptedFHIRBundle {
    + iv : string {12 bytes Hex}
    + authTag : string {16 bytes Hex}
    + encryptedData : string {Base64}
    + keyId : string
    + recordHash : string
  }
}

PatientReference "1" o-- "0..*" RecordReference : indexed by patientRefHash
PatientReference "1" o-- "0..*" Consent : grants
ProviderReference "1" <-- "0..*" Consent : grantee
RecordReference "1" <..> "1" EncryptedFHIRBundle : cryptographic proof anchor
PatientReference "1" o-- "0..*" AccessEvent : immutable audit trail
PatientReference "1" o-- "0..*" EmergencyAccessEvent : break-glass override log
@enduml
```

11.3 Sequence: Consent-Gated Access & Decryption

This document models the end-to-end flow of dynamic consent policy evaluation, tamper verification, off-chain decryption, and immutable on-chain access logging.

📊 UML Sequence Diagram (Mermaid)

```
sequenceDiagram
    autonumber
    actor Clinician as 🩺 Clinician (Requester)
    participant Gateway as 🌐 MedraLink API Gateway
    participant ConsentAgent as 🤖 ConsentAgent
    participant Fabric as ⛓️ Hyperledger Fabric (Ledger)
    participant Vault as 🗄️ Custodial Hospital Vault

    Clinician->>Gateway: GET /records/:id?consentId=...&purpose=treatment<br/>(X-User-
Role: Clinician)
    Gateway->>ConsentAgent: evaluateConsent(consentId, patientRef, requester, scope,
purpose)
    ConsentAgent->>Fabric: Query Consent(consentId) & Provider(requester)
    Fabric-->>ConsentAgent: Consent Record & Provider X.509 Status

    alt Consent Invalid / Expired / Purpose Mismatch
        ConsentAgent-->>Gateway: FAIL_CLOSED (Denied: EXPIRED or SCOPE_MISMATCH)
        Gateway->>Fabric: LogAccess(logId, requestId, requester, recordId, DENIED)
        Gateway-->>Clinician: 403 Forbidden (FHIR OperationOutcome)
    else Consent Valid & Active
        ConsentAgent-->>Gateway: CONSENT_AUTHORIZED (Scope: [AllergyIntolerance,
MedicationRequest])
        Gateway->>Fabric: Query RecordReference(recordId)
        Fabric-->>Gateway: recordHash (SHA-256 anchor) & storagePointer

        Gateway->>Vault: Fetch Ciphertext Payload(storagePointer)
        Vault-->>Gateway: Encrypted Ciphertext + IV + AuthTag

        Note over Gateway: Verify SHA-256(ciphertext) == recordHash anchor
        alt Ciphertext Hash Mismatch (Tamper Detected)
            Gateway-->>Clinician: 500 Tamper Integrity Violation
        else Cryptographic Hash Matches
            Note over Gateway: AES-256-GCM Decrypt using Envelope DEK
            Gateway->>Fabric: LogAccess(logId, requestId, requester, recordId, GRANTED)
            Fabric-->>Gateway: Transaction Committed (Block #N)
            Gateway-->>Clinician: 200 OK + Plaintext FHIR R4 Bundle
        end
    end
```

📋 PlantUML Source Code (`sequence_consent.puml`)

```
@startuml
autonumber
skinparam style strictuml
skinparam sequenceMessageAlign center
```

```
actor "Clinician\n(Doctor)" as Doctor
participant "REST API\nGateway" as Gateway
participant "ConsentAgent\n(Policy Engine)" as Agent
participant "Hyperledger\nFabric 2.5" as Fabric
participant "Hospital\nStorage Vault" as Vault

Doctor -> Gateway : GET /records/{id}?consentId=...&purpose=treatment\n[X-User-Role:
Clinician]
activate Gateway

Gateway -> Agent : evaluateConsent(consentId, requester, scope, purpose)
activate Agent

Agent -> Fabric : Query State: CONSENT_{id} & PROV_{requester}
activate Fabric
Fabric --> Agent : Consent Object & Provider Status
deactivate Fabric

alt Consent Revoked / Expired / Scope Mismatch
    Agent --> Gateway : Status: DENIED (FAIL_CLOSED)
    Gateway -> Fabric : LogAccess(logId, requester, recordId, status="DENIED")
    Gateway --> Doctor : 403 Forbidden [FHIR OperationOutcome]
else Consent Valid & Active
    Agent --> Gateway : Status: GRANTED
    deactivate Agent

    Gateway -> Fabric : Query State: REC_{recordId}
    activate Fabric
    Fabric --> Gateway : recordHash {SHA-256 Anchor}, storagePointer
    deactivate Fabric

    Gateway -> Vault : Fetch Ciphertext(storagePointer)
    activate Vault
    Vault --> Gateway : Ciphertext + IV + AuthTag
    deactivate Vault

    note over Gateway : Assert SHA-256(Ciphertext) == recordHash\n(Tamper Proof Check)

    alt Hash Mismatch
        Gateway --> Doctor : 500 Error [Ciphertext Integrity Violation]
    else Hash Validated
        note over Gateway : AES-256-GCM Envelope Decrypt\nProduce HL7 FHIR R4 Bundle
        Gateway -> Fabric : LogAccess(logId, requester, recordId, status="GRANTED")
        activate Fabric
        Fabric --> Gateway : Tx Committed [Block Height #N]
        deactivate Fabric
        Gateway --> Doctor : 200 OK [Decrypted FHIR R4 Bundle]
    end
end

deactivate Gateway
@enduml
```

11.4 Sequence: Emergency Break-Glass & Post-Hoc Audit

This document models the life-safety emergency break-glass sequence, triage agent validation, 60-minute token dispensation, and post-hoc DGHS auditor review.

📊 UML Sequence Diagram (Mermaid)

```
sequenceDiagram
    autonumber
    actor ER as 🚨 Emergency Doctor
    participant Gateway as 🌐 API Gateway
    participant Triage as 🤖 EmergencyTriageAgent
    participant Fabric as ⛓️ Hyperledger Fabric
    actor Auditor as 🛡️ DGHS Compliance Auditor
    participant AuditAgent as 🤖 AuditAgent

    Note over ER: Unconscious Trauma Patient Arrives
    ER->>Gateway: POST /emergency/invoke<br/>{patientRefHash, reasonCode, vitals, GCS:
7, MAP: 55}
    Gateway->>Triage: evaluateEmergencyProtocol(vitals, reasonCode, clinicianMfa)

    Triage->>Triage: Calculate Shock Index (HR/SBP) & GCS (7 < 8 = ESI Level 1)
    Triage-->>Gateway: EMERGENCY_VALIDATED (Issue 60-Min Token)

    Gateway->>Fabric: InvokeEmergencyAccess(emergencyId, clinicianId, patientRef,
reasonCode, 60min)
    Fabric-->>Gateway: State Committed: EMERGENCY_{id} [PENDING_REVIEW]
    Gateway-->>ER: 201 Created (60-min Break-Glass Token + Emergency Allergy Summary)

    Note over ER: Emergency Resuscitation Complete (60 mins pass)

    Note over Auditor, AuditAgent: Post-Hoc Regulatory Audit Cycle
    AuditAgent->>Fabric: Scan Ledger Blocks for PENDING_REVIEW Emergency Events
    Fabric-->>AuditAgent: Return emergencyId, reasonCode, vitals payload
    AuditAgent->>AuditAgent: Cross-reference hospital ESI logs & trauma admission
registry
    AuditAgent-->>Auditor: Flag Dossier with Evidence Findings

    Auditor->>Gateway: POST /emergency/review<br/>{emergencyId, reviewStatus:
"APPROPRIATE", findingsNote: "..."}
    Gateway->>Fabric: ReviewEmergencyAccess(emergencyId, auditorId, APPROPRIATE,
findingsHash)
    Fabric-->>Gateway: State Committed: EMERGENCY_{id} [REVIEWED]
    Gateway-->>Auditor: 200 OK (Cryptographic Non-Repudiation Audit Receipt)
```

📋 PlantUML Source Code (`sequence_emergency.puml`)

```
@startuml
autonumber
skinparam style strictuml
skinparam sequenceMessageAlign center
```

```
actor "Emergency Clinician\n(Trauma ED)" as ER
participant "REST Gateway" as Gateway
participant "EmergencyTriageAgent" as Triage
participant "Hyperledger Fabric\nLedger" as Fabric
participant "AuditAgent\n(Forensic Scanner)" as Scanner
actor "DGHS Auditor\n(Compliance)" as Auditor

== Phase 1: Pre-Authorized Break-Glass Invocation ==
ER -> Gateway : POST /emergency/invoke\n{patientRef, reasonCode="UNCONSCIOUS_TRAUMA",
vitals}
activate Gateway

Gateway -> Triage : evaluateEmergencyTriage(vitals, reasonCode)
activate Triage
note over Triage : Evaluate Glasgow Coma Scale (GCS <= 8)\nVerify Mean Arterial Pressure
(MAP < 65)\nAssert ESI Level 1 Critical Protocol
Triage --> Gateway : Token Dispensed [60-minute Expiry]
deactivate Triage

Gateway -> Fabric : InvokeEmergencyAccess(emergencyId, clinicianId, patientRef,
reasonCode, expiry)
activate Fabric
Fabric --> Gateway : State: EMERGENCY_{id} [Status: PENDING_REVIEW]
deactivate Fabric

Gateway --> ER : 201 Created [Break-Glass Token + Allergy Intolerance Alert]
deactivate Gateway

== Phase 2: Post-Hoc Compliance Forensic Audit ==
Scanner -> Fabric : Parse Blocks for PENDING_REVIEW Events
activate Scanner
activate Fabric
Fabric --> Scanner : Unreviewed Emergency Records
deactivate Fabric

Scanner -> Scanner : Cross-Verify Hospital EHR SIEM Feeds
Scanner --> Auditor : Present Forensic Dossier & Recommendation
deactivate Scanner

Auditor -> Gateway : POST /emergency/review\n{emergencyId, reviewStatus="APPROPRIATE",
findings}
activate Gateway

Gateway -> Fabric : ReviewEmergencyAccess(emergencyId, auditorId, status, findingsHash)
activate Fabric
Fabric --> Gateway : State: EMERGENCY_{id} [Status: REVIEWED, reviewStatus: APPROPRIATE]
deactivate Fabric

Gateway --> Auditor : 200 OK [Immutable Regulatory Audit Receipt]
deactivate Gateway

@enduml
```

11.5 State Machine: Consent & Emergency Lifecycles

This document models the lifecycle states, transitions, guard conditions, and fail-closed termination of Consent Tokens and Emergency Break-Glass events on the Hyperledger Fabric ledger.

📊 1. Consent Token State Machine (Mermaid)

```
stateDiagram-v2
    [*] --> DRAFT : Patient Selects Scope & Grantee

    DRAFT --> ACTIVE : GrantConsent(consentId, patientRef, grantee, scope, purpose,
expiry)

    state ACTIVE {
        [*] --> VERIFIED : Access Request Received
        VERIFIED --> DECRYPT_ALLOWED : Grantee == Requester && Purpose Valid && Time <
Expiry
        VERIFIED --> ACCESS_DENIED : Grantee Mismatch || Purpose Mismatch
        ACCESS_DENIED --> [*] : LogAccess(DENIED)
        DECRYPT_ALLOWED --> [*] : LogAccess(GRANTED)
    }

    ACTIVE --> EXPIRED : CurrentTime > ExpiryTimestamp [Temporal Invariant]
    ACTIVE --> REVOKED : RevokeConsent(consentId) by Patient [PDPO 2025 Right to Revoke]

    EXPIRED --> [*] : Terminal Fail-Closed (No Decryption)
    REVOKED --> [*] : Terminal Fail-Closed (No Decryption)
```

📊 2. Emergency Break-Glass State Machine (Mermaid)

```
stateDiagram-v2
    [*] --> INITIATED : Trauma Patient Admitted (GCS <= 8)

    INITIATED --> ACTIVE_60MIN : InvokeEmergencyAccess(emergencyId, reasonCode, vitals)

    state ACTIVE_60MIN {
        [*] --> EMERGENCY_UNLOCKED : Emergency Clinician Reads Allergies/Meds
        EMERGENCY_UNLOCKED --> [*]
    }

    ACTIVE_60MIN --> PENDING_REVIEW : 60 Minutes Elapse (Token Expired)

    state PENDING_REVIEW {
        [*] --> AUDIT_INVESTIGATION : DGHS Auditor Scans Ledger & EHR Feeds
        AUDIT_INVESTIGATION --> APPROPRIATE : Justified Clinical Indication
        AUDIT_INVESTIGATION --> INAPPROPRIATE : Unjustified Break-Glass Attempt
    }

    APPROPRIATE --> ARCHIVED_CLEARED : ReviewEmergencyAccess(APPROPRIATE, findingsHash)
    INAPPROPRIATE --> DISCIPLINARY_ACTION : ReviewEmergencyAccess(INAPPROPRIATE,
findingsHash) -> Flag BMDC

    ARCHIVED_CLEARED --> [*]
```

```
    DISCIPLINARY_ACTION --> [*]
```

📋 PlantUML Source Code (`state_machine.puml`)

```
@startuml
skinparam state {
  BackgroundColor #F8FAFC
  BorderColor #0F2A44
  ArrowColor #0D9488
}

title MedraLink Consent & Emergency State Machine

state "Consent Token Lifecycle" as ConsentLifecycle {
  [*] --> Active : GrantConsent()\n[Scope, Purpose, Expiry]

  Active --> Active : RequestAccess()\n[Valid Grantee & Purpose]
  Active --> Expired : CurrentTime >= ExpiryTimestamp\n[Temporal Guard]
  Active --> Revoked : RevokeConsent()\n[Patient Invocation]

  Expired --> [*] : Access Denied\n(Fail-Closed)
  Revoked --> [*] : Access Denied\n(Fail-Closed)
}

state "Emergency Break-Glass Lifecycle" as EmergencyLifecycle {
  [*] --> ActiveEmergency : InvokeEmergencyAccess()\n[ReasonCode, Trauma Vitals]

  ActiveEmergency --> PendingReview : 60-Minute Expiry Elapsed

  state PendingReview {
    [*] --> AuditorReview
    AuditorReview --> Appropriate : Clinical Protocol Justified
    AuditorReview --> Inappropriate : Unauthorized / Fake Emergency
  }

  Appropriate --> ClosedAppropriate : ReviewEmergencyAccess()\n[Findings Hash Anchored]
  Inappropriate --> ClosedSanctioned : Disciplinary Referral to BMDC\n[Regulatory Non-
Repudiation]

  ClosedAppropriate --> [*]
  ClosedSanctioned --> [*]
}
@enduml
```

11.6 Activity: Agentic Multi-Agent DAG Workflow

This document models the Directed Acyclic Graph (DAG) workflow scheduling, parallel agent execution, and blockchain state settlement handled by `MedraLinkOrchestrator`.

📊 UML Activity Diagram (Mermaid)

```
flowchart TD
    Start([Start Clinical Intake Event]) --> DAG_Planner[MedraLinkOrchestrator: Build
DAG Execution Plan]

    subgraph Step1["Step 1: Terminology & Data Transformation"]
        RawClinicalNotes[Raw Unstructured Clinical Notes / Lab Feed]
        DAG_Planner --> FHIRAgent[🤖 FHIRAgent: Semantic Normalization]
        RawClinicalNotes --> FHIRAgent
        FHIRAgent --> Bindings[Extract SNOMED-CT / LOINC / RxNorm Codes]
        Bindings --> BuildBundle[Construct 6-Resource HL7 FHIR R4 Bundle]
    end

    subgraph Step2["Step 2: Cryptographic Envelope & Policy Verification"]
        BuildBundle --> Fork((Fork Concurrent Checks))
        Fork --> EncryptService[🔒 EncryptionService: AES-256-GCM Envelope Encryption]
        Fork --> ConsentAgent[🤖 ConsentAgent: Dynamic PDPO 2025 Policy Evaluator]

        EncryptService --> GenDEK[Generate DEK & Compute recordHash = SHA256 Ciphertext]
        ConsentAgent --> CheckRules{Validate Active Consent, Temporal Bounds & Scope}
    end

    CheckRules -- Violates Policy --> Deny[⛔ Access Denied / Fail-Closed Gate]
    Deny --> EndFail([End Workflow with Access Logged])

    CheckRules -- Compliant --> Join((Join Verification))
    GenDEK --> Join

    subgraph Step3["Step 3: Off-Chain Storage & On-Chain Settlement"]
        Join --> UploadVault[Upload Ciphertext to Hospital Vault s3://vault/rec.enc]
        UploadVault --> AnchorLedger[⛓️ Fabric Settlement: CreateRecordReference
recordId, recordHash]
        AnchorLedger --> AuditTrigger[🤖 AuditAgent: Trigger Asynchronous SIEM Anomaly
Scan]
    end

    AuditTrigger --> EndSuccess([Intake Completed Successfully with Cryptographic
Receipt])
```

📋 PlantUML Source Code (`activity_dag.puml`)

```
@startuml
skinparam activity {
  BackgroundColor #F8FAFC
  BorderColor #0F2A44
  ArrowColor #0D9488
}
```

```
start
:Client triggers DAG Workflow Pipeline\n[e.g. CLINICAL_INTAKE_AND_RECORD_ANCHOR];

:MedraLinkOrchestrator initializes DAG plan\nand allocates Working Session Memory;

partition "Stage 1: Semantic Normalization" {
  :FHIRAgent parses clinical observations and medications;
  :Map raw terms to SNOMED-CT, LOINC, and RxNorm ontologies;
  :Assemble validated HL7 FHIR R4 Bundle;
}

fork
  partition "Stage 2A: Cryptographic Envelope" {
    :Generate AES-256 Data Encryption Key (DEK);
    :Encrypt FHIR bundle with AES-256-GCM;
    :Compute recordHash = SHA256(Ciphertext);
  }
fork again
  partition "Stage 2B: Dynamic Consent Policy" {
    :ConsentAgent queries Fabric World State;
    :Assert temporal validity (currentTime < expiry);
    :Assert purpose binding and clinical scope allowlist;
  }
end fork

if (Consent Policy Satisfied?) then (yes)
  partition "Stage 3: Off-Chain & On-Chain Settlement" {
    :Upload ciphertext payload to custodial hospital repository;
    :Anchor RecordReference(recordId, recordHash) to Hyperledger Fabric;
    :Emit RecordAnchored chaincode event to SSE stream;
    :AuditAgent checks hash continuity and updates audit baseline;
  }
  :Return 200 OK + Cryptographic Transaction Receipt;
  stop
else (no)
  :Log Access Violation (FAIL_CLOSED) on Ledger;
  :Emit life-safety alert or rejection OperationOutcome;
  stop
endif
@enduml
```

11.7 Deployment: 4-Org Consortium Topology

This document models the physical and containerized node topology of the MedraLink 4-Organization permissioned consortium network.

📊 UML Deployment Diagram (Mermaid)

```
graph TB
    subgraph ClientTier["1. User & Client Tier"]
        Browser["🖥️ React.js 18 Web SPA<br/>(Tailwind CSS, Fluid Typography)"]
        MobileApp["📱 Patient Health Card App<br/>(Web Crypto, QR Pseudonym)"]
    end

    subgraph APITier["2. API Gateway & Agent Layer"]
        Gateway["🌐 MedraLink REST Gateway (Node.js 20)<br/>- Port: 3001<br/>- SSE Event
Stream<br/>- Express RBAC & FHIR Adapters"]
        AgentEngine["🤖 Agentic AI Multi-Agent Engine<br/>- ConsentAgent,
FHIRAgent<br/>- EmergencyTriageAgent, AuditAgent<br/>- MedraLinkOrchestrator"]
    end

    subgraph FabricConsortium["3. Hyperledger Fabric 2.5 Consortium Network"]
        subgraph Org1["Org1MSP (BSMMU Hospital A)"]
            Peer1["peer0.org1.medralink.com<br/>- Endorser + Anchor<br/>- Go Chaincode
Engine"]
            Couch1[("CouchDB State DB 1")]
            CA1["ca.org1.medralink.com<br/>(X.509 MSP)"]
            Peer1 --- Couch1
        end

        subgraph Org2["Org2MSP (Evercare Hospital B)"]
            Peer2["peer0.org2.medralink.com<br/>- Endorser<br/>- Go Chaincode Engine"]
            Couch2[("CouchDB State DB 2")]
            CA2["ca.org2.medralink.com<br/>(X.509 MSP)"]
            Peer2 --- Couch2
        end

        subgraph Org3["Org3MSP (National Health Gateway)"]
            Peer3["peer0.org3.medralink.com<br/>- Endorser & Router"]
            Couch3[("CouchDB State DB 3")]
            Peer3 --- Couch3
        end

        subgraph OrgAuditor["OrgAuditorMSP (DGHS Regulatory)"]
            PeerAuditor["peer0.auditor.medralink.com<br/>- Read-Only Audit Replica"]
            CouchAuditor[("CouchDB Audit DB")]
            PeerAuditor --- CouchAuditor
        end

        subgraph OrderingCluster["Raft Ordering Service"]
            Orderer1["orderer1.medralink.com<br/>(Raft CFT Consenter)"]
            Orderer2["orderer2.medralink.com<br/>(Raft CFT Consenter)"]
            Orderer3["orderer3.medralink.com<br/>(Raft CFT Consenter)"]
        end
    end

    subgraph StorageTier["4. Off-Chain Custodial Storage Repositories"]
```

```
        VaultA["🗄️ Hospital A EMR Storage<br/>(AES-256-GCM Encrypted FHIR)"]
        VaultB["🗄️ Hospital B EMR Storage<br/>(AES-256-GCM Encrypted FHIR)"]
    end

    Browser -->|HTTP / SSE| Gateway
    MobileApp -->|HTTP REST| Gateway
    Gateway <--> AgentEngine
    Gateway -->|gRPC / Fabric SDK| Peer1
    Gateway -->|gRPC / Fabric SDK| Peer2
    Gateway -->|HTTPS Encrypted Payloads| VaultA
    Gateway -->|HTTPS Encrypted Payloads| VaultB

    Peer1 --- Orderer1
    Peer2 --- Orderer1
    Peer3 --- Orderer2
    PeerAuditor --- Orderer3
```

📋 PlantUML Source Code (`deployment_topology.puml`)

```
@startuml
skinparam node {
  BackgroundColor #F8FAFC
  BorderColor #0F2A44
}
skinparam database {
  BackgroundColor #0F172A
  BorderColor #14B8A6
  FontColor #F8FAFC
}

package "Client Presentation Layer" {
  node "Client Browser" as Client {
    artifact "React.js 18 SPA\n[Vite, Tailwind, WebCrypto]" as SPA
  }
}

package "Microservices Gateway Layer" {
  node "API Gateway Server (Node.js 20)" as GatewayNode {
    component "Express REST Gateway\n[Port 3001, SSE, RBAC]" as Gateway
    component "Agentic AI Engine\n[5 Autonomous Agents & DAG]" as AgentEngine
  }
}

package "Hyperledger Fabric 2.5 Consortium Network (Channel: medralink-main)" {
  node "Org1MSP (Hospital A - BSMMU)" as Org1 {
    component "peer0.org1.medralink.com\n[Go Chaincode medralink-cc]" as Peer1
    database "CouchDB State 1" as DB1
    Peer1 -> DB1
  }

  node "Org2MSP (Hospital B - Evercare)" as Org2 {
    component "peer0.org2.medralink.com\n[Go Chaincode medralink-cc]" as Peer2
    database "CouchDB State 2" as DB2
    Peer2 -> DB2
  }
```

```
  node "OrgAuditorMSP (DGHS Regulatory)" as OrgAuditor {
    component "peer0.auditor.medralink.com\n[Read-Only Audit Replica]" as PeerAuditor
    database "CouchDB Audit DB" as DBAuditor
    PeerAuditor -> DBAuditor
  }

  node "Raft Ordering Cluster" as OrderingCluster {
    component "3-Node Raft CFT Orderer Cluster\n[orderer1..3.medralink.com]" as Orderers
  }
}

package "Custodial Hospital Repositories (Off-Chain)" {
  node "Hospital A Custodial Cloud" as HospitalACloud {
    database "Encrypted FHIR Vault A\n[AES-256-GCM Ciphertext]" as VaultA
  }
  node "Hospital B Custodial Cloud" as HospitalBCloud {
    database "Encrypted FHIR Vault B\n[AES-256-GCM Ciphertext]" as VaultB
  }
}

SPA --> Gateway : HTTPS / SSE Stream (Port 3001)
Gateway <--> AgentEngine : Inter-Process Memory & Context
Gateway --> Peer1 : gRPC Endorsement (7051)
Gateway --> Peer2 : gRPC Endorsement (8051)
Gateway --> PeerAuditor : gRPC Audit Query (9051)
Gateway --> VaultA : Envelope Encrypted Storage (HTTPS)
Gateway --> VaultB : Envelope Encrypted Storage (HTTPS)

Peer1 --> Orderers : Broadcast Committed Transactions
Peer2 --> Orderers : Broadcast Committed Transactions
Orderers --> PeerAuditor : Deliver Ordered Ledger Blocks
@enduml
```

Chapter 12: High-Resolution Mock Screenshots & Presentation Catalog

This gallery provides high-definition (1920×1080) mock screenshots of the **MedraLink Decentralized Healthcare Interoperability & Audit Provenance Platform** for use in:

- 📊 **Competition Slide Decks & Pitch Presentations** (BCOLBD 2026 Student Track)
- 🖼️ **Academic Posters & Banners** (A1/A0 Print Formats)
- 📑 **Technical Reports, Whitepaper Addenda & Demos**

All image files are located in **[`prototype/docs/screenshots/`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/)**.

🖼️ High-Resolution Mock Screenshots Catalog

1. Patient Consent Governance & Health Vault

- **File:** [`01_Patient_Portal_Consent_Vault.png`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/01_Patient_Portal_Consent_Vault.png)
- **Description:** Patient self-sovereign consent management dashboard. Shows granular scope selection (`AllergyIntolerance`, `MedicationRequest`, `Observation`), purpose binding (`treatment`, `diagnostic`), active/expired consent tokens, and immutable access history.
- **Key Talking Point:** Demonstrates strict compliance with Bangladesh Personal Data Protection Ordinance (PDPO 2025) and patient consent self-sovereignty without raw PII on ledger.

2. Authorized Clinician Consent-Gated Record Decryption

- **File:** [`02_Clinician_Portal_Decrypted_FHIR.png`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/02_Clinician_Portal_Decrypted_FHIR.png)
- **Description:** Clinician record access portal. Demonstrates on-chain consent verification, cryptographic hash integrity checks (`recordHash == SHA256(ciphertext)`), AES-256-GCM envelope decryption, and dual clinical/JSON views.
- **Key Talking Point:** Proves zero-trust off-chain storage architecture where clinical payloads are only decrypted when authorized by an active smart contract consent token.

3. Emergency Break-Glass Life-Safety Protocol

- **File:** [`03_Emergency_Break_Glass_Portal.png`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/03_Emergency_Break_Glass_Portal.png)
- **Description:** Emergency trauma override portal. Displays Glasgow Coma Scale (GCS) and Mean Arterial Pressure (MAP) inputs, 60-minute time-boxed emergency break-glass token dispensation, and life-threatening allergy alerts.
- **Key Talking Point:** Solves delayed trauma care for unconscious patients while preventing unauthorized snooping through time-boxed tokens and mandatory audit review.

4. DGHS Regulatory Compliance & Forensic Audit Dashboard

- **File:** [`04_Auditor_Portal_Compliance_Dashboard.png`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/04_Auditor_Portal_Compliance_Dashboard.png)
- **Description:** Directorate General of Health Services (DGHS) auditor portal. Shows emergency break-glass review queue (`APPROPRIATE` vs `INAPPROPRIATE`), cryptographic findings hash anchoring, and live blockchain block explorer.

- **Key Talking Point:** Provides regulatory non-repudiation and automated referral to the Bangladesh Medical & Dental Council (BMDC) for any unverified break-glass abuse.

5. Consortium Administration & Identity Onboarding

- **File:** [`05_Consortium_Admin_Portal.png`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/05_Consortium_Admin_Portal.png)
- **Description:** Consortium administrator portal. Shows X.509 healthcare provider registration (`BSMMU`, `Evercare`, `National Health Gateway`), certificate serial hashing, and synthetic patient pseudonymization (`SHA256(HealthID || DOB || Salt)`).
- **Key Talking Point:** Enforces 4-Organization permissioned network governance with zero storage of national identity numbers (NID) or raw biometric templates on blockchain.

6. Agentic AI Studio — Master DAG Orchestrator

- **File:** [`06_Agentic_AI_Studio_DAG_Orchestrator.png`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/06_Agentic_AI_Studio_DAG_Orchestrator.png)
- **Description:** Autonomous AI Multi-Agent orchestration studio. Features 3 canonical clinical scenarios, dynamic Directed Acyclic Graph (DAG) planner, and visual multi-tier execution trace.
- **Key Talking Point:** Demonstrates how 5 specialized autonomous agents coordinate clinical intake, semantic normalization, policy checks, and blockchain settlement in <300ms.

7. Agentic AI — FHIR Semantic Normalization Sandbox

- **File:** [`07_Agentic_AI_FHIR_Normalization.png`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/07_Agentic_AI_FHIR_Normalization.png)
- **Description:** `FHIRAgent` sandbox. Translates unstructured clinical notes and laboratory feeds into structured HL7 FHIR R4 bundles with SNOMED-CT, LOINC, and RxNorm ontology codes.
- **Key Talking Point:** Bridges the interoperability gap between Bangladesh's 5,000+ disparate legacy EMR systems and the unified national standard.

8. Agentic AI — Trauma Triage & Break-Glass Evaluator

- **File:** [`08_Agentic_AI_Trauma_Triage.png`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/08_Agentic_AI_Trauma_Triage.png)
- **Description:** `EmergencyTriageAgent` sandbox. Evaluates incoming physiological vitals, calculates Shock Index and Emergency Severity Index (ESI Level 1), and issues 60-min cryptographic break-glass tokens.
- **Key Talking Point:** Uses algorithmic vital validation to prevent unauthorized physician break-glass abuse while ensuring instant access during genuine emergencies.

9. Agentic AI — Forensic Ledger & SIEM Scanner

- **File:** [`09_Agentic_AI_Forensic_Audit.png`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/09_Agentic_AI_Forensic_Audit.png)
- **Description:** `AuditAgent` sandbox. Continuously parses Hyperledger Fabric ledger blocks and hospital SIEM telemetry, detects anomalous access patterns, and constructs evidence dossiers for DGHS auditors.
- **Key Talking Point:** Transforms static audit logs into proactive AI-assisted regulatory oversight.

10. Interactive 9-Step Demo Tour Modal

- **File:** [`10_Interactive_Demo_Tour_Modal.png`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/10_Interactive_Demo_Tour_Modal.png)

- **Description:** Interactive 9-step guided walkthrough for competition judges, covering the entire lifecycle from consortium bootstrap to forensic audit clearance.
- **Key Talking Point:** Highlights end-to-end usability and test-driven prototype maturity for jury evaluations.

💡 Best Practices for Slides & Poster Presentations

33. **Aspect Ratio:** All images are native **16:9 (1920×1080)**, optimized for modern widescreen slide presentations (PowerPoint, Google Slides, Keynote) and 2-column or 3-column academic poster layouts.
34. **Crop Suggestions:**

- For **slide feature callouts**, crop specific cards (e.g., the "Consent-Gated Vault" or "DAG Execution Trace") to highlight technical architecture.
- For **poster centerpieces**, use [`06_Agentic_AI_Studio_DAG_Orchestrator.png`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/06_Agentic_AI_Studio_DAG_Orchestrator.png) or [`02_Clinician_Portal_Decrypted_FHIR.png`](file:///home/tr/Downloads/MedraLink/prototype/docs/screenshots/02_Clinician_Portal_Decrypted_FHIR.png) to showcase the rich dark-mode UI aesthetics.

35. **Contrast:** The sleek obsidian dark theme (`#0F172A`) with emerald, sky-blue, and amber status accents provides maximum contrast on 4K projectors and glossy banner prints.